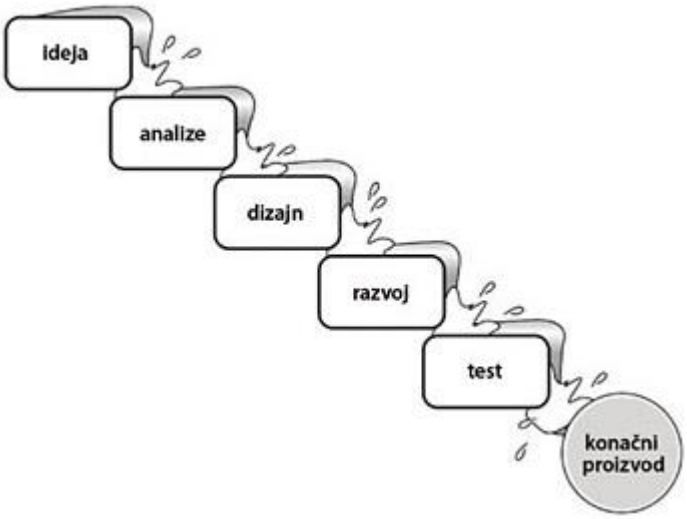
~ Razvoj višeslojnih aplikacija u elektroenergetici - ispitna pitanja~

1. Postupci razvoja softvera. Kaskadni i iterativni procesi. – poređenje



Kaskadni procesi

2. UML dijagrami. Namena, načini korišćenje, vrste.

- UML (Unified Modeling Language - objektni jezik za modelovanje) je skup grafičkih notacija zasnovanih na jedinstvenom metamodelu. UML je vizuelni jezik koji se koristi za: vizuelizaciju, specifikovanje, konstruisanje i dokumentovanje softverskih sistema.

- Namena: 1. komunikacija - programera, njegovih kolega iz tima, programera i klijenata
  2. razumevanje - jasnija slika o sistemu
  3. preciznost - dobar opis koda sa aspekta objektno orijentisanog dizajna

- Način korišćenja:
  1. Izrada skica
     - Najčešće korišćena
     - Na dva načina:
       a. Direktan razvoj (forward engineering) – UML dijagram se crta pre pisanja koda
          - Cilj: lakše saopštavanje ideja i alternativa u toku diskusije, brza analiza koda čiji razvoj dugo traje
       b. Povratna analiza (reverse engineering) – dijagram se pravi prema postojećem kodu (razumevanje)
          - Objašnjenje kako radi deo sistema – koriste se samo delovi klasa
     - Neformalno i dinamičko – brzo i timski – na tabli
     - Dokumentacija – gde je bitna informacija, a ne potpunost
     - Često se ne koriste sva pravila UML-a
  2. opis projekta
     - Potpuni dijagrami – detaljan opis sistema
     - Jednoznačno direktno pisanje koda
     - Detaljne informacije o projektu – povratna analiza
     - Složniji alati za izradu UML projekata
       - Često integrisani u razvojna okruženja
     - Nejasna u potpunosti granica između projekata i skica

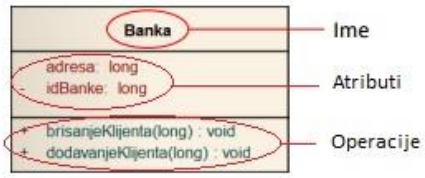
3. programski jezik
- Kada se celokupan sistem opisuje UML dijagramima – UML se koristi kao programski jezik
- UML dijagrami se neposredno prevode u izvršni kod (UML je izvorni kod)
- Modelovanje softvera – dijagrami potpuno opisuju softverske elemente

- Notacija - skup grafičkih elemenata – sintaksa jezika za modelovanje (npr. za dijagrame klasa definiše kako se predstavljaju elementi i koncepti kao što su klasa, asocijacija i kardinalnost)
-Metamodeli - dijagram (najčešće dijagram klasa) – definiše koncepte jezika

- UML dijagrami: 1. strukturni dijagrami
    a. dijagram klasa
    b. kompozitno strukturni dijagram
    c. dijagram komponenti
    d. dijagram raspoređivanja
    e. dijagram objekata
    f. dijagram paketa
  2. dijagrami ponašanja
    a. dijagram aktivnosti
    b. dijagram interakcije
      b.1. sekvencijalni dijagram
      b.2. komunikacioni dijagram
      b.3. pregled interakcija dijagram
      b.4. vremenski dijagram
    c. use case dijagram
    d. dijagram stanja
  - najčešće korišćeni: Dijagrami klasa (klase, odlike, veze), d.interakcije (kombinacija d.sekvenci I aktivnosti), use case d.(interakcija korisnika I sistema), d.aktivnosti (proceduralno I paralelno ponašanje), d.stanja(kako događaji menjaju objekat), d.deployment-a(fizička organizacija sistema)

3. Blokovi za izgradnju UML-a. Klase i relacije.
- Blokovi za izgradnju UML-a:
  1. Stvari
     - Strukturne – klasa, interfejs, kolaboracija, use-case funkcije, aktivna klasa, komponenta, čvor
     - Opšte – poruka, stanje
     - Grupišuće – paket
     - Anotacijske - komentar
  2. Relacije : Zavisnost, Asocijacija, Generalizacija, Realizacija
  3. Dijagrami

  - Klasa - skup objekata koji dele iste atribute, operacije, relacije i semantiku
    - grafički se predstavlja pravougaonikom

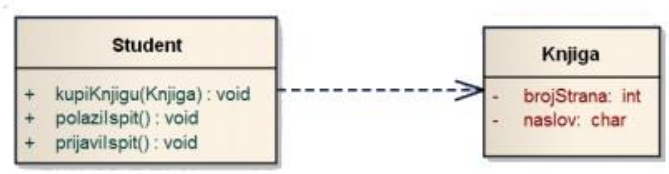


  - Ime klase – niz znakova: jednostavno; sa putanjom (prefiks – pripadnost paketu).
    - Počinje velikim slovom. Ako ima više reči – svaka reč počinje velikim slovom
  - Atributi - imenovana osobina klase bitna za aspekt modelovanje (klasa ih može imati više ili 0)
    - Počinje malim slovom. Ako ima više reči – svaka reč sledeća počinje velikim slovom

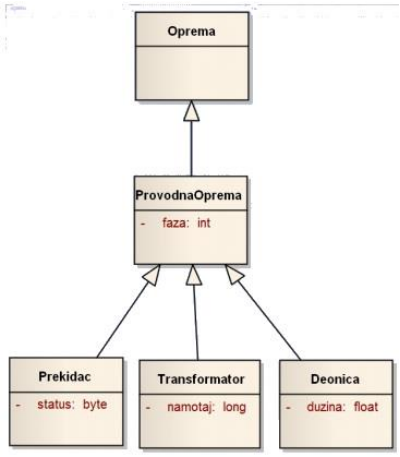
- Operacije - Implementacija nekog servisa
  - Apstrakcija akcije nad objektom
  - Klasa može imati: 0, 1 ili više operacija
  - Počinje malim slovom. Ako ima više reči – svaka reč sledeća počinje velikim slovom
  - Opcioni prikaz atributa i operacija klase

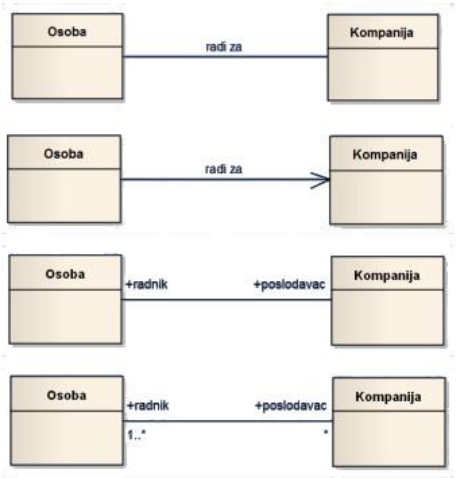
- Relacije:

1. Zavisnost - relacija korišćenja. Relacija koja označava da promena u specifikaciji jedne stvari može uticati na drugu stvar (obrnuto ne važi). Najčešće se koristi za prikaz da jedna klasa koristi drugu kao argument u deklaraciji operacije
   - primer: Klasa Student zavisi od klase Knjiga

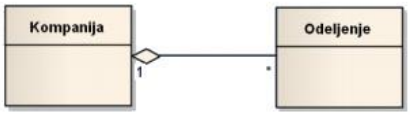


2. Generalizacija - povezuje opštiju klasu sa specifičnijom – natklasa/potklasa (Potklasa obično ima svoje attribute)





3. Asocijacija – strukturna relacija između konkretnih primerak. Povezuje dve klase.
   - Opciona podešavanja: ime asocijacije („radi za“), usmerenje, uloge, višestrukost, grupisanje



4. Dijagrami klasa. Osnovni elementi.

- Najčešće korišćeni dijagrami. Dijagrami klasa (class diagram) opisuje tipove objekata u sistemu i veze između njih. Objekti imaju odlike (feature): svojstva (properties) i operacijama (operations). Svojstva - karakteristike klase (njena polja) mogu biti: atributi I asocijacije

Atributi

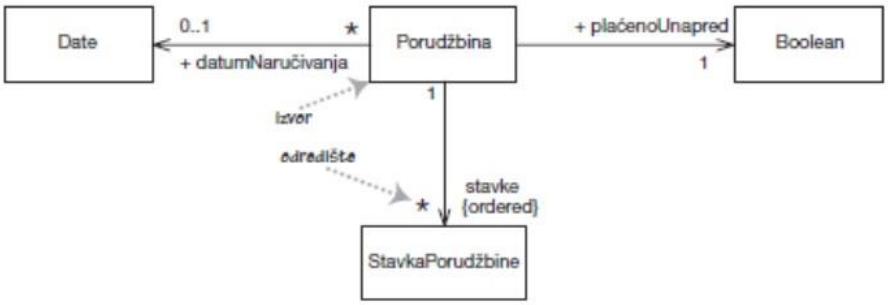
-Zapis atributa

vidljivost ime: tip kardinalnost = podrazumevana_vrednost {opis_svojstva}

- primer: ime: String [1] = “Bez naslova” {readOnly} (obavezno je samo ime)
  - vidljivost (da li je atribut javni (+) ili privatni (-)), ime (kako se zove atribut u klasi), tip (vrsta objekta kaji se smešta u atribut), kardinalnost (jedan ili više objekata u atributu), podrazumevana_vrednost (vrednost atributa u novom objektu), opis_svojstva–opis (pokazuje da li je dozvoljeno menjanje svojstva ({readOnly}))

Asocijacije - Drugi način oznake svojstva (poseduje većinu informacija kao i atribut)

- Označava se punom linijom između dve klase
  - ime svojstva je odredišnom kraju
  - odredišna klasa određuje tip svojstva
  - kardinalnost može biti na oba kraja



Kardinalnost (multiplicity)

- Na koliko se objekata odnosi svojstvo:
  - 1 (jedna porudžbina mora imati tačno jednog kupca) – jedna vrednost
  - 0..1 (za klijentsku firmu može biti zadužen poseban predstavnik, ali ne mora) – opcioni br.
  - * (kupac ne mora poslati porudžbinu, ali ne postoji ni gornja granica)- veći broj vrednosti
  - Ako je značajan redosled porudžbina u asocijaciji treba dodati opis {ordered}, ako se dozvoljavaju duplikati {nonunique}, za standardne osobine se stavlja {unordered} i {unique}

Predstavljanje svojstva u program

public class StavkaPorudzbine…

public int Kolicina; public Novac cena; public Porudzbina porudzbina; public Proizvod proizvod;

- Metode get i set omogućuju čitanje i izmenu atributa
- Klijenti ne vide postojanje atributa

Dvosmerne asocijacije

- Dvosmerna asocijacija – par svojstava iz različitih klasa koja su povezana kao uzajamno inverzna (strelice na oba kraja)
- II način označevanja asocijacije: Upotreba glagolskog oblika za imenovanje asocijacije (nema strelica)

I način

II način

Operacije

- Aktivnosti koje klasa ume da obavi - metode klase

  vidljivost ime (lista_parametara) : tip_rezultata {opis_svojstva}

  -vidljivost - da li je atribut javni (+) ili privatni (-)

  - ime - kako se zove operacija u klasi

  -lista_parametara - lista parametara operacije

  Sintaksa: smer ime: tip = podrazumevana_vrednost

  - ime, tip i podrazumevana_vrednost imaju isto značenje kao kod atributa

  - smer – da li je parametar ulazni (in)-default, izlazni (out) ili ulazno-izlazni (inout)

  - tip_rezultata – vrsta objekta kaji se smešta u atribut

  - opis_svojstva – ukazuje na svojstva operacije

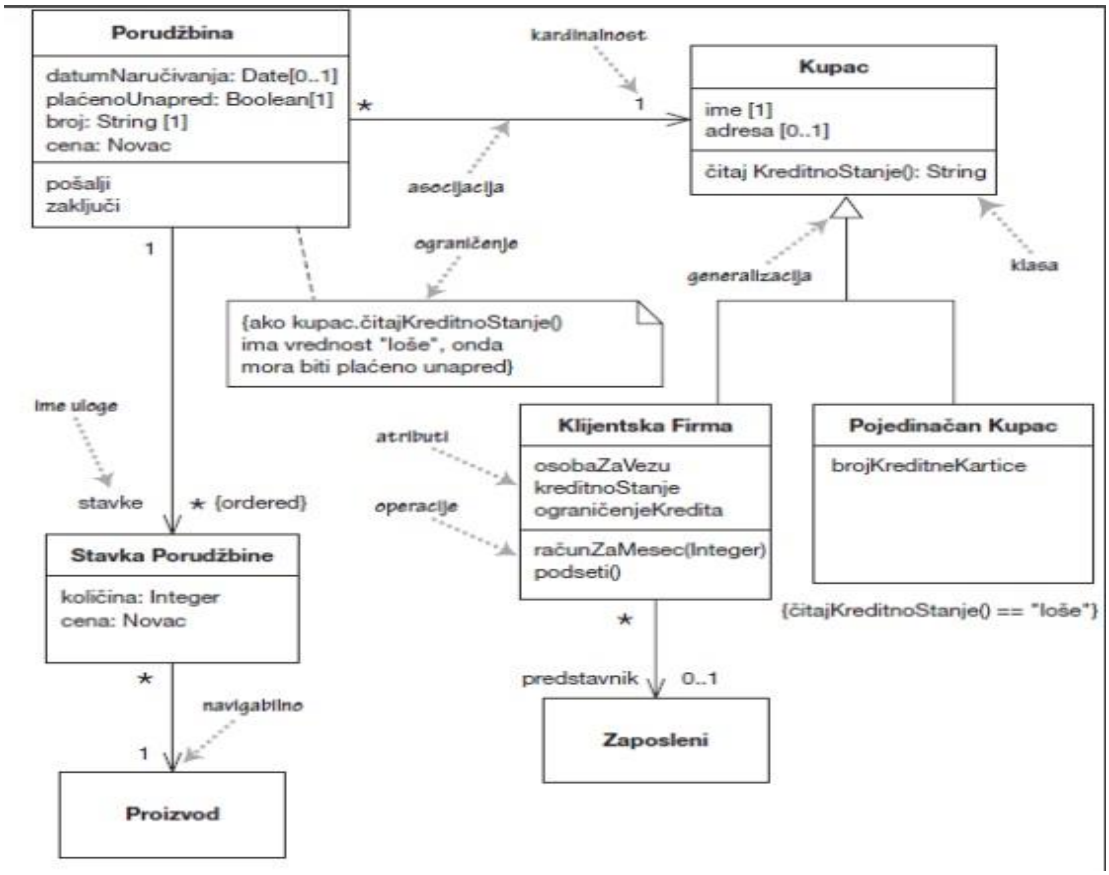
- Primer operacije klase Account:

  +stanjeNa (datum:Datum) : Novac

- upit (query)– operacija čitanja
- modifikator ili komanda – operacija menjanja stanja
- Operacija ≠ metoda
- Operacija se primenjuje nad objektom (deklaracija procedure)
- Metoda je telo procedure (drugačija je ako postoji polimorfizam)

Generalizacija

- Konceptualno podtipovi imaju sve osobine osnovnog tipa - Generalizacija
- U softveru se realizuje nasleđivanjem
- Klijentska firma i pojedinačni kupci su podtipovi klase Kupac
- zamenljivost

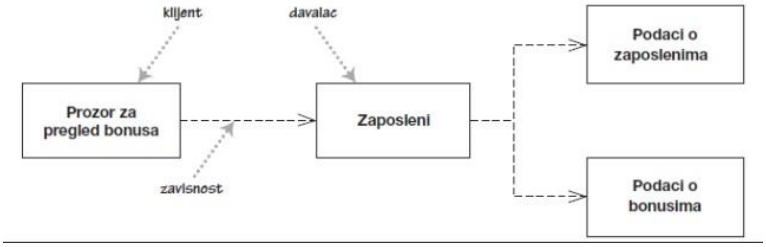


Napomene I komentari - Mogu se pojaviti na svakoj vrsti dijagrama. Nezavisni su od drugih elemenata ili spojeni isprekidanom linijom sa elementima koje objašnjavaju

Zavisnost

-Ako promene definicije jednog objekta (supplier) mogu izazvati promene drugog objekta (klijent)

-Zavisnosti: jedna klasa šalje poruku drugoj, jedna sadrži drugu, objekat jedne klase prosleđuje objekat druge klase kao parametar neke operacije



-Reči za opisivanje zavisnosti:

Rezervisana reč	Značenje
<<call>>	Izvor poziva operaciju odredišta.
<<create>>	Izvor pravi instance odredišta.
<<derive>>	Izvor je izveden iz odredišta.
<<instantiate>>	Izvor je instanca odredišta. Zapazite da, ako je izvor veze klasa, onda je to instanca klase klasa, što znači da je odredište metaklasa (engl. *metaclass*).
<<permit>>	Odredište dozvoljava izvoru da pristupa privatnim karakteristikama.
<<realize>>	Izvor je realizacija specifikacije ili interfejsa koji su definisani odredištem (69. strana).
<<refine>>	Preciziranje detalja (engl. *refinement*) ukazuje na odnos između različitih nivoa značenja. Na primer, izvor može biti klasa nastala tokom projektovanja, a odredište odgovarajuća klasa napravljena tokom analize.
<<substitute>>	Izvor može zameniti odredište (45. strana).
<<trace>>	Koristi se za evidentiranje, na primer, zahteva upućenih klasama ili veza između promena jednog modela i drugih promena.
<<use>>	U realizaciji izvora koristi se odredište veze.

Ograničenja

- Ne mogu se prikazati osnovnim elementima
- Porudžbina se navodi: „40 braon proizvoda, 40 plavih proizvoda i 40 crvenih proizvoda“ a ne 120 p.
- Ova pravila se navode unutar vitičastih zagrada {} korišćenjem proizvoljnog jezika

Kada treba koristiti dijagrame klasa?

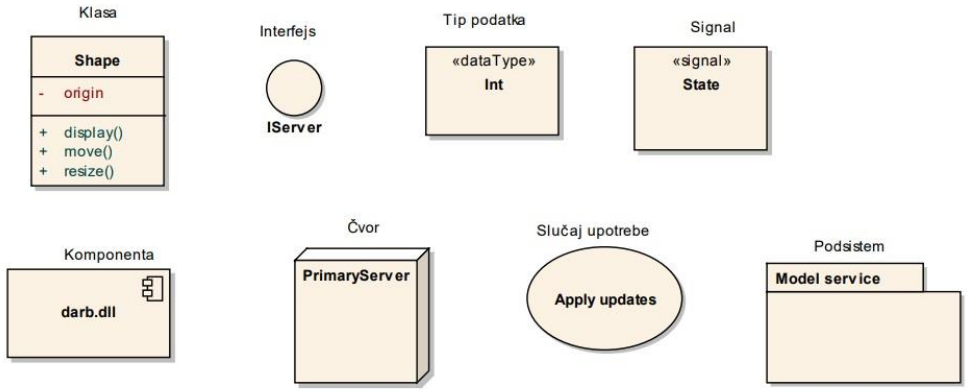
- Oni su osnovni dijagram – stalno se koriste
- Preporuke:
  - Ne treba pokušavati koristiti sve elemente, krenuti od jednostavnih: klase, asocijacije, atributa, generalizacije i ograničenja

- Konceptualni dijagrami klasa su korisni za istraživanje poslovne terminologije – izostaviti softver iz rasprave i koristiti jednostavnu notaciju
- Ne treba crtati sve modele, već samo one najvažnije. Bolje je koristiti nekoliko dijagrama i stalno ih ažirirati, nego puno zaboravljenih i zastarelih modela
- Prilikom kreiranja dijagrama klasa treba voditi računa i o strukturi i o opisu ponašanja

5. Dijagrami klasa. Napredni elementi

- Rezervisane reči - Pojašnjavaju značenje simbola
  - Neke se pišu u vitičastim zagradama, npr. {abstract}
  - neke se pišu između znakova « i », npr. «interface»

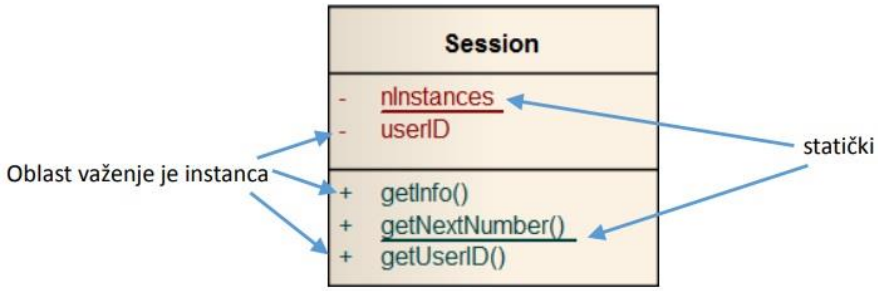
- Klasifikatori



Odgovornost (responsibility)

- Kratke rečenice koja opisuje značajnije funkcije objekta: Aktivnosti, Znanje koje poseduje, Značajne odluke koje donosi

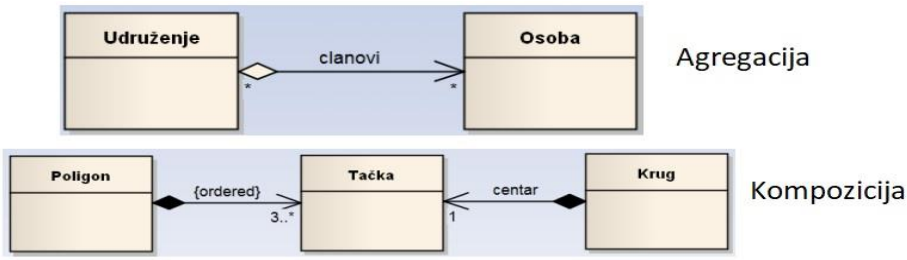
Statičke operacije i atributi - Odnose se na celu klasu, a ne na njenu instancu – statički
- Ekvivalentni su statičkim članovima u programskim jezicima



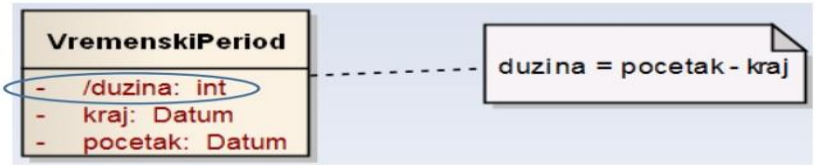
Vidljivost - Pojednostavljeno – svaka klasa ima javne i privatne elemente
- UML koristi oznake:
  + javni (public)
  - privatni (private)
  ~ paketni (package)
  # zaštićeni (protected)
- Programski jezici – različita tumačenja
- Najčešće se koriste samo + i –

Agregacija i kompozicija

- Agregacija predstavlja odnos celina-deo (npr. automobil - točkovi)
- Kompozicija – svaka instanca mora pripadati samo jednom vlasniku (može biti više pot. vlasnika)



Izvedena svojstva (derived properties) - Mogu se izračunati na osnovu drugih svojstava
- Izvedeno svojstvo označava se kosom crtom

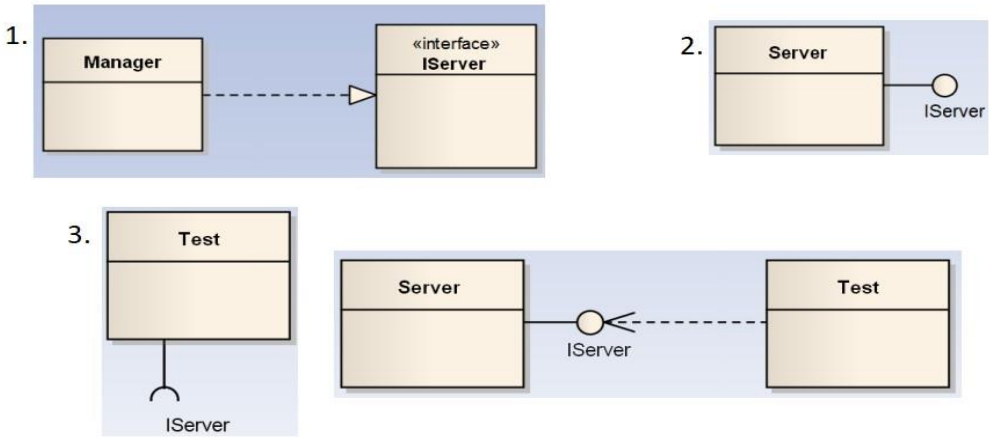


Interfesi i apstraktne klase

- Objekti apstraktne klase se ne mogu napraviti – već samo primerak podklase
- Apstraktna klasa sadrži jednu ili više apstraktnih operacija (nije realizovana, već deklarisana)
- U UML-u apstraktne klase i operacije su napisane iskošeno ili kao {abstract}
- Interfejs je klasa koja nije realizovana (ima sve apstraktne karakteristike)
- Označava se sa «interface»
- Klasa uspostavlja dve vrste veze sa interfejsima:
  - realizuje interfejs (zamenljiva je sa interfejsom)
  - zahteva interfejs (koristi instancu interfejsa)

Interfejs na UML dijagramima

1. Označavanje interfejsa kao klase (klasa Manager realizuje interfejs IServer)
2. Ili samo kružićem
3. Ako neka klasa Test zavisi od interfejsa



Referentni objekti i vrednosni objekti

- Referentni objekti (reference objects) su oni koji u softveru treba da budu predstavljeni klasom i imaju identitet (provera da li dve reference pokazuju na isti objekat se utvrđuje upoređivanjem njihovih identiteta)
- Vrednosni objekti (value objects), npr. datum, više vrednosnih objekata predstavlja jedan stvarni

Asocijacija opisane kvalifikatorom

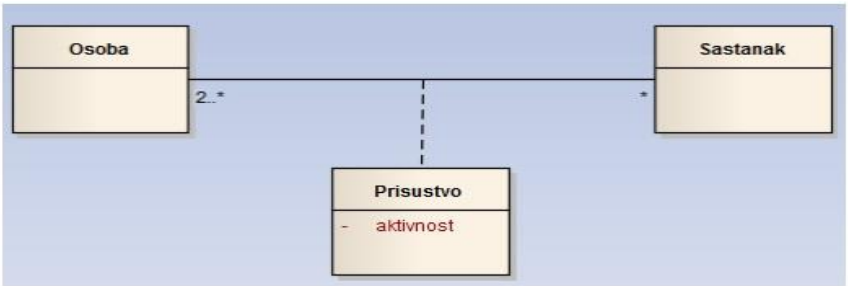
- Ekvivalentna je raznim pojmovima u programskim jezicima:
  - Asocijativnim nizovima (associative array)
  - Mapama (maps)
  - Heš – tabelama (hashes)
  - Rečnicima (dictionaries)

Višestruka i dinamička klasifikacija

- Klasifikacija je veza između objekta i njegovog tipa
- Jednostruka klasifikacija – jedan objekat pripada jednom tipu
- Višestruka klasifikacija – jedan objekat može biti opisan pomoću nekoliko tipova, koji ne moraju biti povezani nasleđivanjem
- Višestruko nasleđivanje ≠ višestruka klasifikacija
- Višestruko nasleđivanje – jedan tip može imati više nadtipova, svaki objekat mora pripadati jednom tipu
- Višestruka klasifikacija – jedan objekat pripada većem broju tipova (ne mora se definisati poseban tip)

Klasa asocijacija

- Omogućuje da se asocijacijama dodaju atributi, operacije koje se čuvaju u jednoj klasi
- Informacija koliko je osoba bila aktivna – atribut aktivnost u klasi Prisustvo

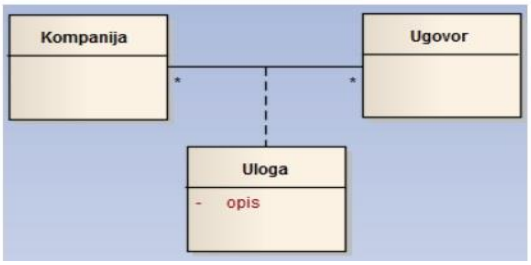


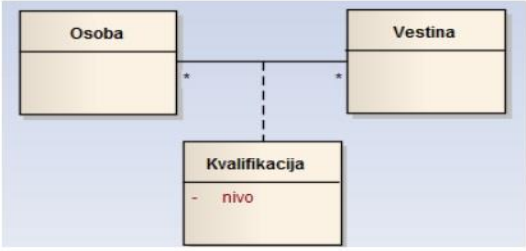
Klasa asocijacija kao potpuna klasa

- Ograničenje: može postojati samo jedan primerak klase

Klasa asocijacija - primer

- Može postojati samo jedna kvalifikacija za jednu veštinu
- Ne može jedna osoba imati različite kvalifikacije za jednu veštinu
- Kompanija ima različite uloge u ugovoru
- Ograničenje – između dva objekta – samo jedna klasa asocijacije
- Preporuka: koristiti klasu Uloga kao potpunu klasu

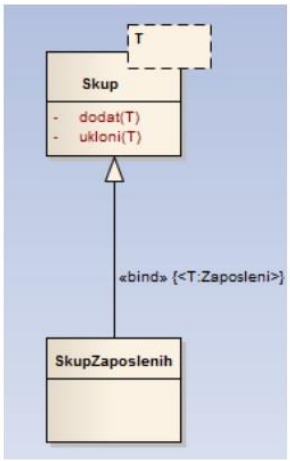




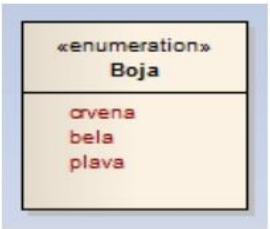
Šablonska (parametrizovana) klasa

I način

II način



Nabrojive liste (Enumerations) - Prikazuju se sa rezervisanom rečju enumeration



Aktivna klasa

- Ima instance od kojih svaka izvršava svoju programsku nit i upravlja njom
- Primer: komandno okruženje koje čita ulazne komandne objekte i izvršava komande u svojoj programskoj niti

6. Dijagrami sekvence.

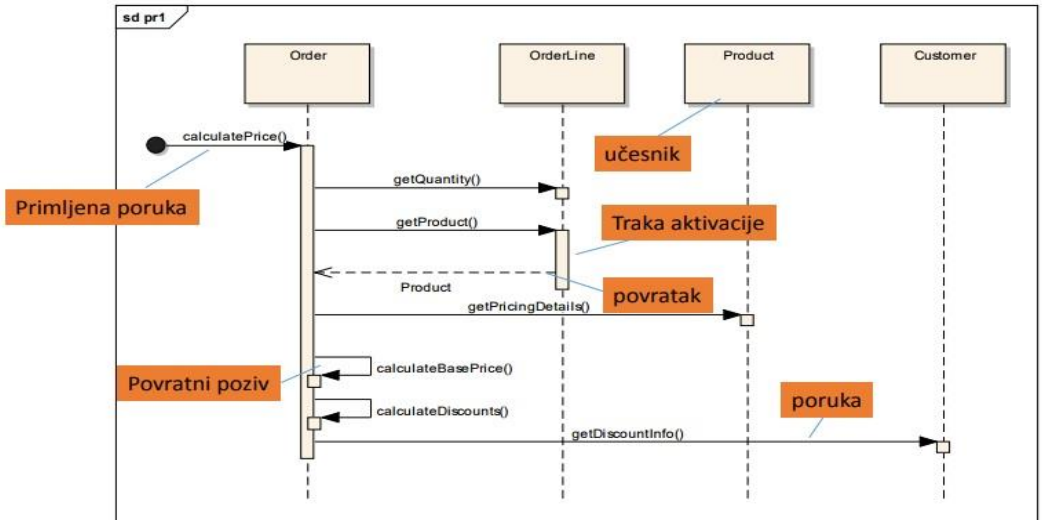
• Najčešći dijagram interakcije je dijagram sekvence
• Prikazuje jedan scenario koji obuhvata više objekata i poruka koje oni razmenjuju za dati slučaj korišćenja
• Opisuje interakciju između učesnika (objekat) prikazanog pomoću vertikalne linije života i poruka koje se čitaju odozgo nadole
• Jednostavni su i lako razumljivi

Karakteristike

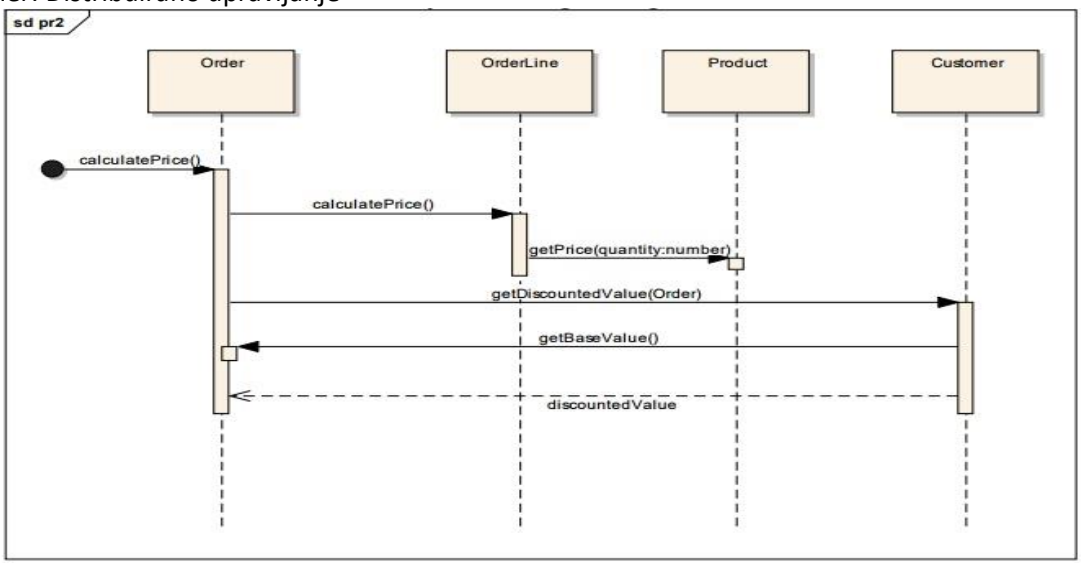
- Svaki pravougaonik predstavlja određeni objekat i sadrži ime klase
- Opciono može pre imena klase sadržati i ime objekta
- Vertikalna linija označava vek trajanja objekta
- Horizontalne linije između dve vertikalne označavaju slanje poruke (poziv metode) iz jednog objekta drugome
- Nekada se povratna vrednost eksplicitno prikazuje

Elementi dijagrama: Učesnici – objekti, Poruke, Linija života, Traka aktivnosti, Primeljena poruka

Primer: centralizovano upravljanje

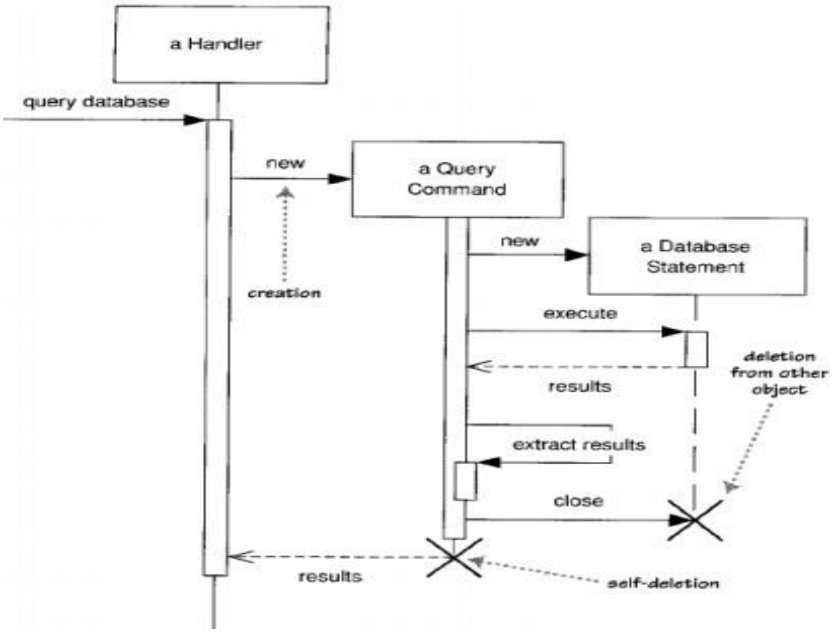


Primer: Distribuirano upravljanje



Kreiranje i brisanje učesnika

- Kreiranje - strelica poruke sa nazivom „new“ ili Create
- Brisanje – označava se sa X.
- Kada jedan učesnik briše drugog šalje mu poruku koja ulazi u X
- Ako učesnik briše samog sebe navodi se X bez primljene poruke

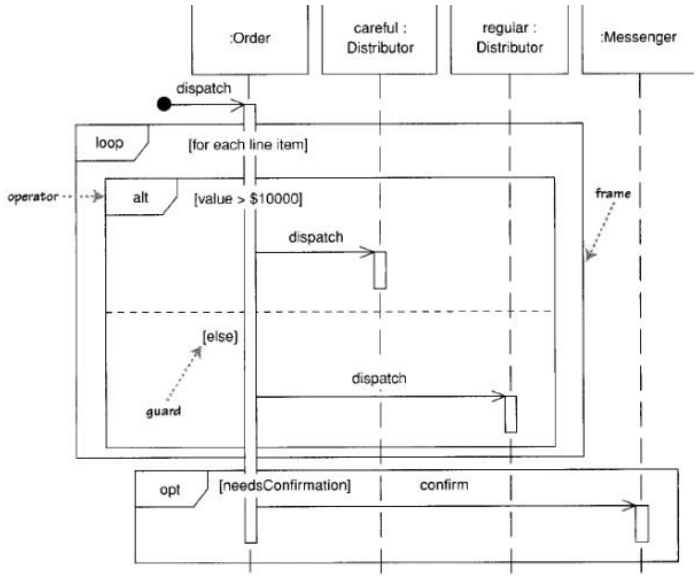


Petlje i uslovi

- Ovi dijagrami nisu najzgodniji za logiku upravljanja
- Bolje koristiti dijagrame aktivnosti ili napisati kod
- Opisuju se okvirima interakcije (interaction frames)
- Sastoje se iz jednog ili više fragmenata kojima se može pridružiti uslov
- Dodaju se operatori u okvire, npr. loop za petlju sa jednim fragmentom, opt za uslov sa jednim fragmentom

Primer-petlje, uslovi

```
procedure dispatch
  foreach (lineitem)
    if (product.value > $10K)
      careful.dispatch
    else
      regular.dispatch
    end if
  end for
  if (needsConfirmation) messenger.confirm
end procedure
```

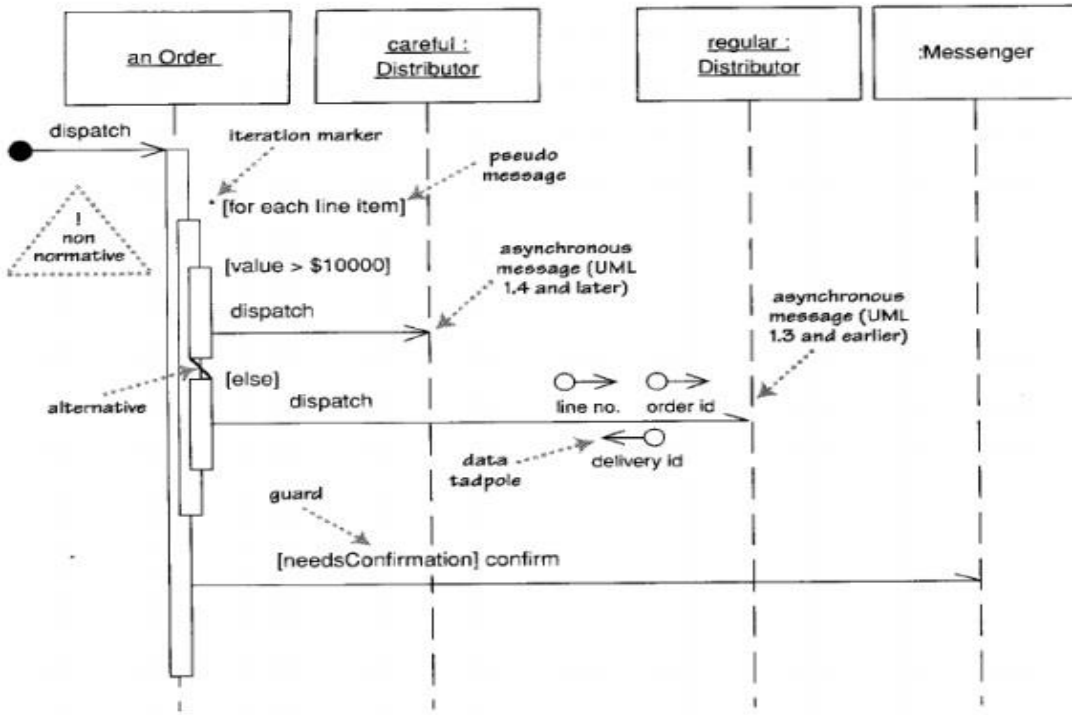


Operatori okvira interakcije

-alt - Alternativni izbor između više fragmenata (u zavisnosti od uslova)
-opt - Opcioni – fragment se izvršava samo ako je uslov ispunjen (alt sa jednim slučajem)
-par Paralelno izvršavanje svih fragmenata
-loop Petlja – fragment se može izvršiti više puta (u zavisnosti od uslova)
-region Fragment u kome se ne može istovremeno izvršavati više niti
-neg Fragment pokazuje pogrešnu interakciju
-ref Referenca na interakciju iz drugog dijagrama
-sd Fragment dijagrama sekvence (uokviri ceo dijagram)

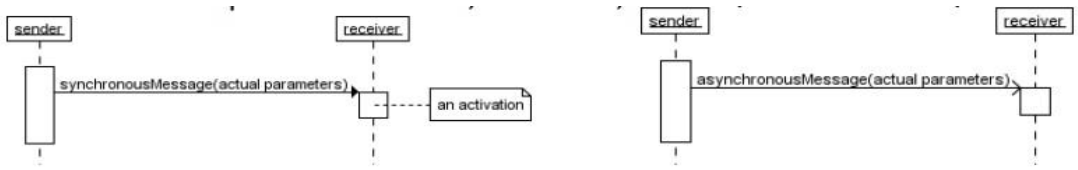
Starije konvencije

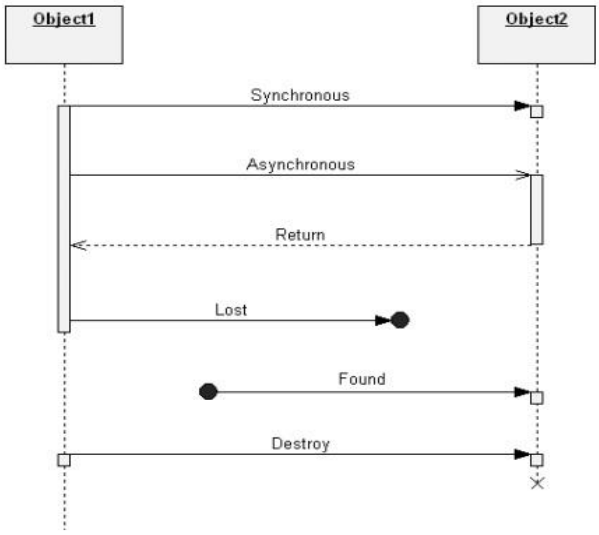
- Znak * za oznaku iteracije
- Opis iteracije u uglastim zagradama
- Uslov je izraz napisan između uglastih zagrada
- Strelica sa lopticom – ukazuje na kretanje podataka



Sinhrone i asinhrone poruke

- Popunjeni vrh strelice – sinhrona poruka
- Prazan vrh ili prepolovljena strelica (starija verzija) – asinhrona poruka
- Sinhrona poruka – pošiljalac čeka na odgovor
- Asinhrona poruka – pošiljalac ne čeka odgovor (može da nastavi da radi)
- Asinhrone poruke – više niti, brži odziv, otežan proces otkrivanja grešaka





Kada treba koristiti dijagrame sekvence?

- Ako se želi posmatrati ponašanje objekata u okviru jednog slučaja upotrebe
- Uspešno pokazuju kolaboraciju objekata
- Nisu pogodni za precizno definisanje ponašanja objekata (tada koristiti dijagram stanja)
- Ako se proučava više slučajeva upotrebe ili više niti – dijagram aktivnosti
- Ako treba ispitati više mogućih interakcija – CRC kartice
- Druga vrsta dijagrama interakcije – dijagrami komunikacije – pokazuju veze i vremenske dijagrame za prikaz ograničenja

CRC kartice

- CRC (Class-Responsibility-Collaboration) – klase – odgovornosti – saradnje

- Okupi se tim oko stola
- Razmotre se različiti scenariji i izvedu pomoću kartica
- Podižu se aktivne kartice i pomeraju da bi se pokazalo kako objekti šalju poruke i prosleđuju podatke
- Za izradu kartica – značajno prepoznavanje odgovornosti – rečenica u kojoj se napisano šta objekat treba da uradi – njegove aktivnosti, znanje i značajne odluke
- Svaka klasa se opisuje preko malog skupa odgovornosti

7. Dijagrami ponašanja - Dijagrami strukture, slučajeva upotrebe i stanja.

Slučajevi upotrebe

- Način prikupljanja funkcionalnih zahteva sistema
- Opisuju uobičajene interakcije korisnika i sistema
- Obično se prvo opisuje scenario – niz koraka koji opisuju interakciju između korisnika i sistema
- Slučaj upotrebe (use case) je skup scenarija povezanih jednim ciljem korisnika
- Ne postoji opis načina na koji se prikupljaju podaci o slučajevima upotrebe – postoji opis dijagrama
- Najveća vrednost slučajeva upotrebe je u sadržaju, a ne u dijagramu

Učesnici u slučajevima upotrebe

- Učesnik je uloga korisnika u odnosu na sistem (npr. kupac, prodavac, direktor prodaje, rač.sistem,i sl.)
- Jedan učesnik može da izvodu više slučajeva upotreba, a jedan slučaj upotrebe može imati više uč.
- Bolji termin od učesnik je uloga (engl. role)

Primer - kupovina proizvoda na Internetu

Kupac pregleda katalog i dodaje proizvode u korpu. Kada kupac hoće da plati, on daje informacije o načinu dostave i platnoj kartici i potvrđuje kupovinu. Sistem proverava ispravnost podataka o platnoj kartici i potvrđuje prodaju, neposredno, a zatim i putem elektronske pošte.

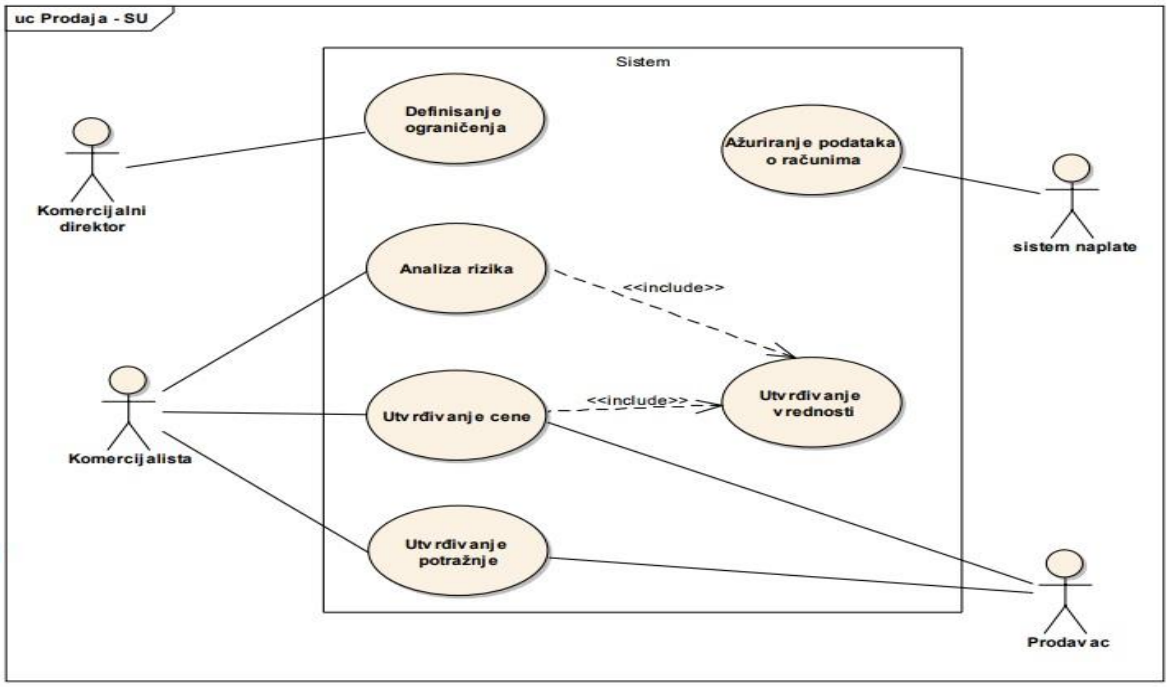
- Ako su podaci o kartici netačni → poseban scenario
- Kupovina redovnog kupca (ne trebaju podaci o dostavi i kartici) – 3. scenario
- Svi scenariji – jedna cilj – kupovina proizvoda

Sadržaj slučaja upotrebe

- Na početku se bira jedan osnovni uspešni scenario i ispisuju se njegovi koraci
- Nakon toga se ispisuju drugi scenariji kao proširenja
- U svakom slučaju upotrebe definiše se glavni učesnik (obično inicijator), ostali učesnici su sporedni uč.
- Svaki korak slučaja upotrebe predstavlja deo interakcije učesnika i sistema
- Korak pokazuje nameru učesnika, a ne način na koji je ostvaruje
- Proširenja navode uslov njihove primene, a nakon toga slede koraci i povezivanje se koracima osnovnog uspešnog scenarija
- Za svaki korak analizirati sve mogućnosti i nepovoljne događaje
- U složenom slučaju upotrebe jedan slučaj upotrebe može biti korak u drugom (koristi se «include» zavisnost)
- U slučajeve upotrebe se mogu dodati:
    - Preduslov (engl. pre-condition) – opisuje šta mora biti ispunjeno da bi sistem dozvolio da počne korišćenje
    - Garancija (engl. guarantee) – opisuje stanje sistema na kraju slučaja upotrebe. Garancije uspeha važe posle uspešnog scenarija, a minimalne garancije posle svakog scenarija
    - Okidač (engl. trigger) – obrađuje događaj koji inicira slučaj upotrebe
- Ne treba praviti komplikovan slučaj upotrebe (bolje je premalo, nego previše)

Dijagram slučajeva upotrebe

- Nisu mnogo dokumentovani
- Važno je tekstualno obraditi slučajeve
- Dijagrami pokazuju:
    - Učesnike
    - Slučajeve upotrebe, i
    - Veze između njih:
        - Koji učesnici izvršavaju koje slučajeve upotrebe
        - Koji slučajevi upotrebe uključuju druge slučajeve upotrebe



Nivoi slučajeva upotrebe

- Sistemski slučaj upotrebe – predstavlja interakciju sa softverom.
- Poslovni slučaj upotrebe– opisuje kako poslovni sistem reaguje na korisnika ili događaj.
- Klasifikacija slučajeva upotrebe po nivoima:
    - Osnovni nivo – predstavlja pojedinačne interakcije između glavnog učesnika i sistema.
        - Daju rezultat za glavnog učesnika.
        - Traju od nekoliko minuta do pola sata.
    - Niži nivo – uključeni u slučajeve osnovnog nivoa.
    - Viši nivo – pokazuju kako se slučajevi upotrebe osnovnog nivoa uklapaju u širi kontekst poslovnih interakcija.
- Poslovni slučajevi upotreba su obično višeg nivoa, a sistemski – nižeg i osnovnog nivoa.
- Najveći broj slučajeva upotreba treba da bude na osnovnom nivou.

Slučajeve upotrebe - preporuke

- Trebalo bi ih skicirati na početku razvojnog postupka
- Detaljnija verzija neposredno pri realizaciji
- Oni treba da budu spoljni pogled na sistem, a ne da opisuju klase sistema
- Tekstualan opis slučaja upotrebe treba da je primaran (često se dijagrami zanemaruju)
- Ne treba ih zakomplikovati

Dijagrami stanja

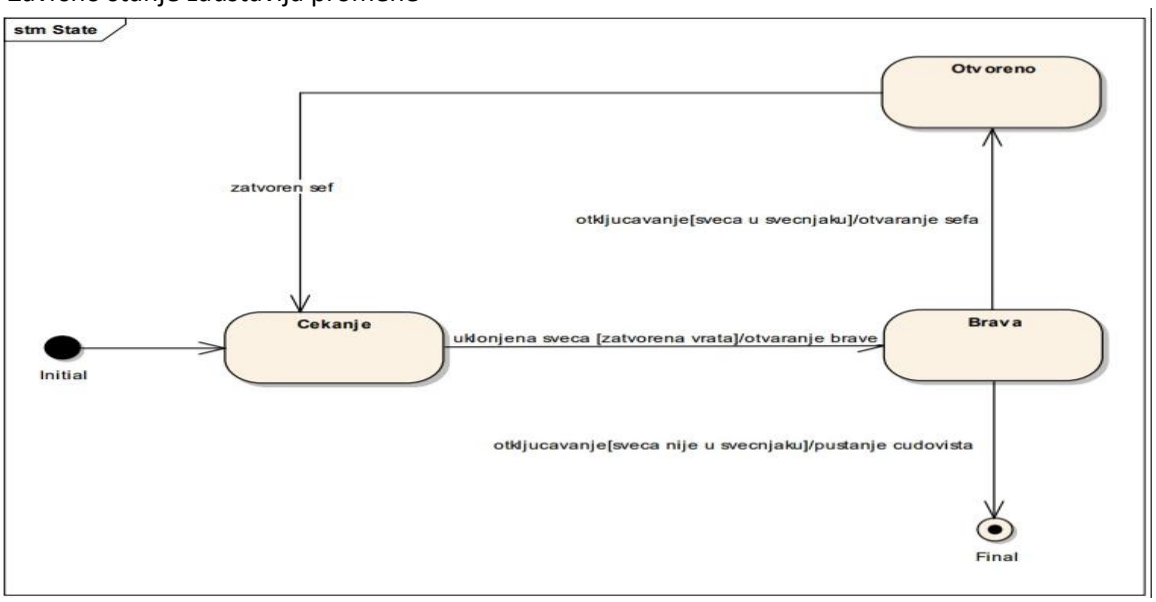
- Početno pseudostanje – vezuje se za nastanak objekta
- Prelaz – ukazuje na pomeranje iz jednog stanja u drugo. Ima obeležje koje se sastoji iz tri dela:
    - Potpis-okidača [uslov]/aktivnost
    - Svi delovi su opcioni
- Potpis-okidača (engl. trigger-signature) – događaj koji omogućava promenu stanja (može obuhvatiti više događaja i parametara)
- Uslov (engl. guard) – logički izraz koji mora biti tačan da bi došlo do prelaza
- Aktivnost (engl. activity) je ponašanje koje se izvršava tokom prelaza

Primer – upravljanje tajnom pregradom u zamku

- Dragocenosti se čuvaju u skrivenom sefu zamka
- Da bi se otkrila brava sefa, mora se ukloniti specijalna sveća iz svećnjaka, ali vrata pri tome moraju biti zatvorena
- Kada se ugleda brava, može da se otključa i otvori sef
- Radi dodatne sigurnosti, sef se otvara samo ako se u svećnjak stavi druga sveća
- Ukoliko lopov otvori bravu a ne stavi drugu sveću, oslobodiće se užasno čudovište koje će ga pojesti

Primer

- Ako se koristi više prelaza za isti događaj – moraju biti međusobno isključivi
- Zanemaren događaj ako nije opisan prelazom (zatvaranje sefa u stanje Čekanje, uklanjanje sveće kada su vrata otvorena)
- Završno stanje zaustavlja promene



Unutrašnje aktivnosti

- Stanja bez prelaza mogu odgovoriti na događaje– unutrašnje aktivnosti
- Primer – stanje tekstualnog polja UI-ja
- Unutrašnja aktivnost – slična povratnom prelazu (vraća se u isto stanje), ali ne pokreću ulazne niti izlazne aktivnosti
- Ulazna aktivnost – izvršava se pri ulazu u stanje
- Izlazna aktivnost – pri napuštanju stanja

Stanja aktivnosti

- Stanje Searching jeste stanje aktivnosti (stanje u kome objekat neprekidno obavlja neki posao)
- Tekuća aktivnost je označena sa do/ i nazvana izvršna aktivnost
- Kada je aktivnost završena – izvršavaju se svi prelazi bez naznačene aktivnosti
- Ako događaj cancel nastane tokom aktivnosti, izvršna aktivnost se prekida i objekat se vraća u stanje

Update Hardware Window

- Izvršna aktivnost
    - Potreban neki period vremena
    - može se prekinuti
- Redovna aktivnost
    - Izvršava se „trenutno“
    - Ne može se prekinuti redovnim događajima

Nadstanja

- Kada više stanja deli zajedničke prelaze – formira se nadstanje

Istovremena stanja

- Stanje se može podeliti u nekoliko horizontalnih dijagrama stanja, koji se izvršavaju istovremeno
- Primer – budilnik koji emituje zvuk sa:
    - CD-a ili Radija.
    - Prikazuje : Tekuće vreme , Vreme alarma
- H - pseudostanje istorije (podešavanje pre uključivanja sata)

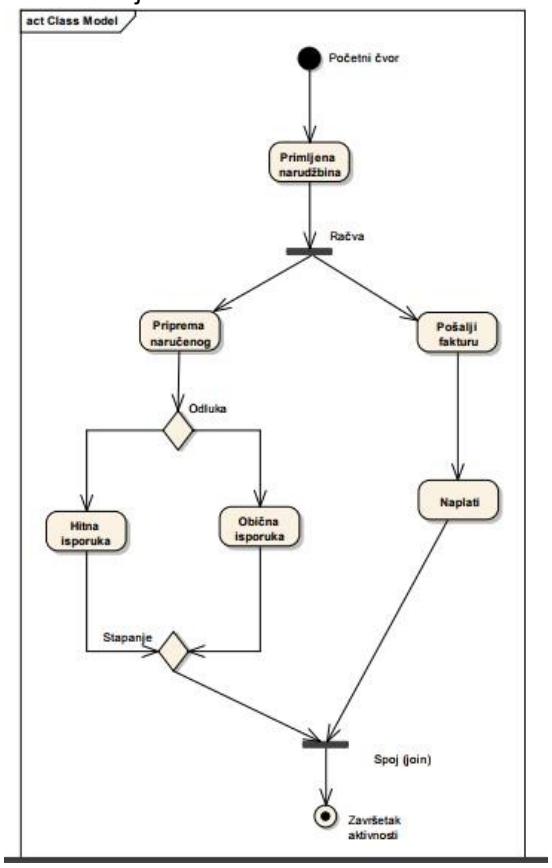
Realizacija dijagrama stanja

1. Pisanje ugnježdenih naredbi
2. Primena obrasca State
3. Izrada tabele stanja

Dijagrami aktivnosti

- Opisivanje toka poslovne logike
- Za razliku od dijagrama toka (flowcharts) mogu prikazati paralelno ponašanje
- Primer – naručivanje robe
- Polazi se od početnog čvora, izvršava se akcija ReceiveOrder, pa dolazi do grananja (paralelno izvršavanje – redosled nije važan)
- Paralelno izvršavanje- sinhronizacija (oznaka join)
- Odluka (decision) ima jedan ulazni i nekoliko uslovnih izlaznih tokova (sa uslovom u [ ])
- Stapanje (merge) ima više ulaznih tokova i jedna izlazni

EA realizacija



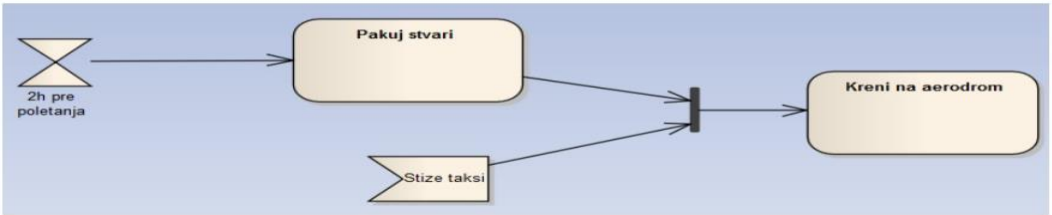
Particije

- Pokazuju koja klasa ili celina izvršava date akcije
- Particija se često naziva i plivačka staza (swim line)

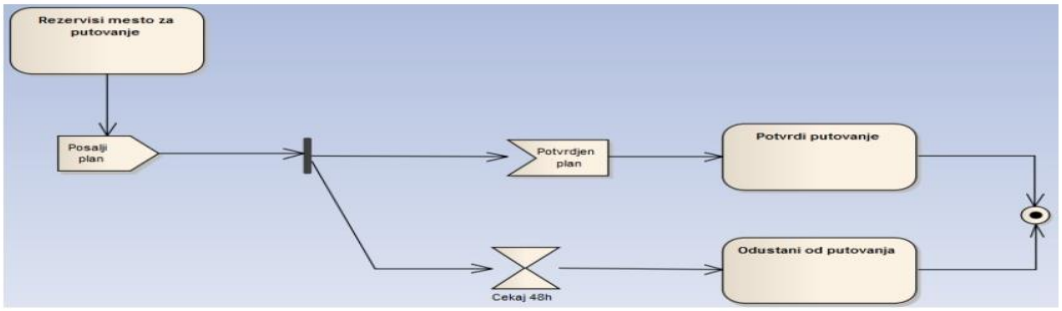
Signali

- Vremenski signal (nastaje zbog protoka vremena)
- Signal ukazuje da aktivnost prima spoljašnji događaj
- Aktivnost osluškuje signale – reaguje po dijagramu
- Signali se mogu primati, ali i slati

Primer 1: Signali



Primer 2: Signali



8. Dijagrami ponašanja - Dijagrami interakcije.

Vrste dijagrama interakcije

- Dijagram sekvence – naglašava redosled i vreme angažovanja učesnika u toku interakcije (određenog scenarija)
- Dijagram komunikacije - naglašava povezanost podataka između učesnika u interakciji
- Dijagram pregleda interakcije – kombinacija dijagrama aktivnosti i dijagrama sekvence
- Vremenski dijagram – prikazuju vremenska ograničenja za jedan ili više objekata

Dijagram komunikacije

- Omogućava proizvoljni raspored učesnika
- Korišćenje decimalnih brojeva za određivanje redosleda poziva
- Umesto brojeva mogu se koristiti i slova, npr. A4 i B2 (različite niti)
- Koriste se za isticanje veze Između učesnika

Dijagram pregleda interakcija

- Može se smatrati kao dijagram aktivnosti koje su predstavljene dijagramima sekvenci
- Primer – ako se želi prikazati zbirni izveštaj o narudžbinama

Vremenski dijagrami

- Više načina prikaza dijagrama (stanja – linije, stanja - oblasti)
- Primer – pumpa i grejač aparata za kafu; 10 sec prođe od uključivanja pumpe do uključivanja grejača; Kada se isprazni posuda za vodu, pumpa se isključuje, a grejač može ostati uključen najviše 15min

9. Dijagrami strukture.

Vrste dijagrama structure: Dijagram klasa, Dijagram komponenata,Dijagram složene structure, Dijagram raspoređivanja, Dijagram objekata, Dijagram paketa

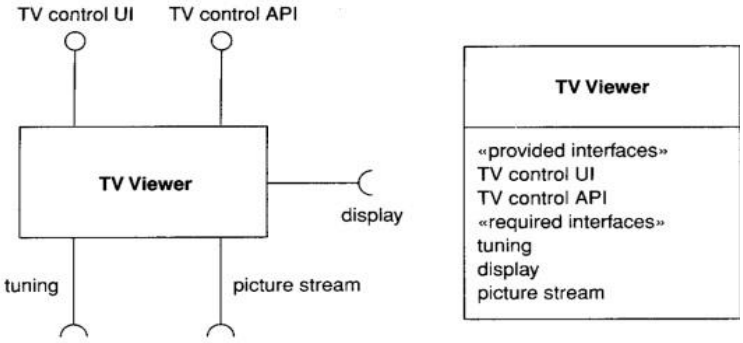
Dijagrami komponenata

- Komponenta <-> obična klasa
- Oznake za komponentu

- Komponente – delovi koji se nezavisno mogu kupovati i nadograđivati
- Podela sistema na komponente – komercijalna, a ne tehnička odluka

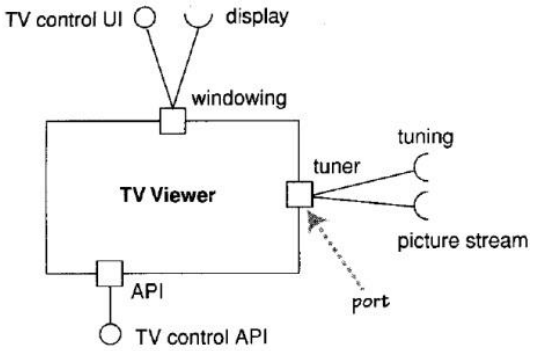
Dijagrami složene strukture

- Omogućuju hijerarhijsko razlaganje klase (složeni objekat se rastavlja na delove)
- TV prijemnik sa interfejsima
- Dva načina prikaza klase



Jedna komponenta sa više priključaka

- Spoljašnjoj strukturi se mogu dodati priključci
- Priključci logički grupišu interfejse



Dijagrami raspoređivanja

- Otkrivaju fizičku organizaju sistema i delove aplikacije koji se izvršavaju na određenim delovima računarskog sistema
- Osnovni elementi su čvorovi
    - Uređaj (fizički element)
    - Izvršno okruženje (program koji sadrži neki drugi program, npr. Operativni sistem, kontejnerski procesi)
    - Sadrže artefakte – fizička manifestacija softvera (najčešće datoteke: izvršne, konfiguracione, podaci, HTML dokumenti)
        - Prikazuju se korišćenjem oznake klase ili ispisivanjem imena unutar čvora
        - Rezervisana reč «artifact»
- Komunikacione putanje između čvorova pokazuju da se između delova sistema odvija komunikacija
- Pogodni su za uvid u izgled sistema

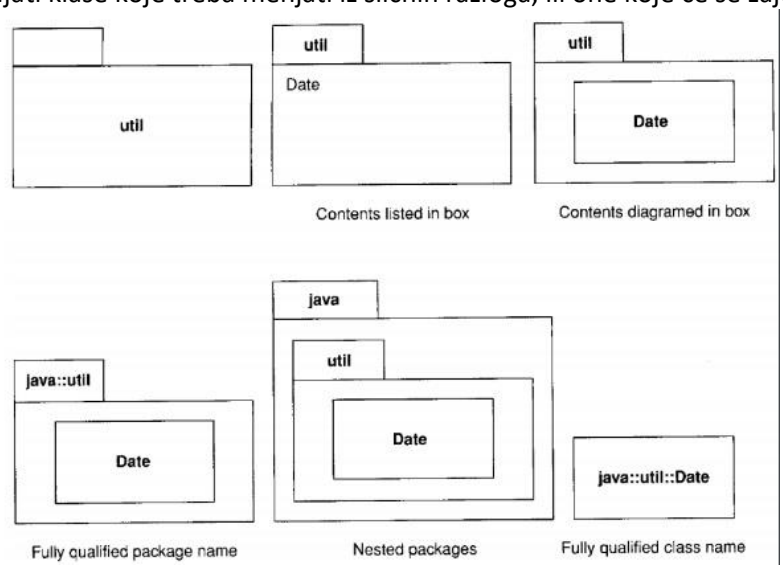
Dijagrami objekata

- Presek objekata sistema u nekom trenutku (uglavnom instance, ne klase)
- Naziva se i dijagram instanci
- Elementi dijagrama su najšešće specifikacije instanci, a ne instance (ne postavljaju se vrednosti obaveznih atributa)
- Može se posmatrati i kao dijagram komunikacije bez poruke
- Koristi se za prikazivanje primera povezanih objekata

Dijagrami paketa

- Paketi – grupisanje elemenata (bilo koje strukture UML-a – ne samo klasa) u celine višeg nivoa (obično veliki broj klasa, dijagrama, use case)
- Hijerarhijska struktura – klasa pripada paketu, paket drugom paketu...
- Svaki paket pripada jednom namespace-u (jedinstveno ime klase u paketu)
- Puno ime klase: Paket:podpaket:Ime
- Na dijagramima je moguće prikazati i sadržaj paketa
- Klase unutar paketa mogu biti javne i privatne (slediti pravila razvojnog okruženja)
- U isti paket treba stavljati klase koje treba menjati iz sličnih razloga, ili one koje će se zajedno koristiti

Primer



Paketi i zavisnosti

- Zavisnosti između paketa – zavisnosti između njihovog sadržaja
- Crtanje paketa-važno za upravljanje strukturom srednjeg ili velikog sistema (najbolje na osnovu koda)
- Primer višeslojne aplikacije

- Dijagrami paketa – mehanizmi za grupisanje delova programa u vreme prevođenja

Analiza
- Jasan tok – isti smer
- Osim dva paketa za pristup podacima – imaju ulogu izolacionog sloja između paketa poslovnog sloja i paketa BazaPodataka
- više paketa zavisi od nekog paketa → stabilniji njegov interfejs (npr. DomenImovine)
- Zavisnosti nisu tranzitivne

Način posmatranja paketa
- Dve vrsta struktura: Slojevita, Poslovna
- Ne prikazuju se pravi paketi
- Problem hijerarhije stvarnih namespace-ova
- Korisni za prikaz strukture složenih aplikacija

Realizacija paketa
- Jedan paket često definiše interfejs koji se može realizovati u više paketa
- Interfejs i njegova realizacija – mogu biti u odvojenim paketima
- Klijentski paket sadrži interfejs realizovan u drugom paketu

Primer: Upravljački objekti korisničkog interfejsa
- Uključivanje/isključivanje
- Prekidači (OnOff) rade sa više objekata (Heater i Light)
- Upravljački objekti pozivaju metode grejača, ali ne zavise od grejača

Modelovanje grupa elemenata - postupak
- Pregledati elemente modela – pronaći grupe konceptualno ili semantički bliskih elemenata
- uključiti takve grupe u pakete
- Za svaki paket odrediti elemente kojima se može pristupiti izvan paketa – označiti ih kao public, ostali su private ili protected
- Povezati zavisne pakete
- U slučaju familije paketa, povezati pomoću generalizacije specijalizovane pakete sa njihovim opštijim delom

10. Stilovi arhitekture. Podela i karakteristike.

- "Stilovi arhitekture su familije sistema – obrasci strukture. Stil određuje rečnik komponenti i konektora koje se mogu koristiti uz niz ograničenja za povezivanje. Ograničenja mogu biti topološka u opisu arhitekture (npr. nema ciklusa) ili u vezi sa izvršnom semantikom."
- Poznavanje stilova obezbeđuje zajednički jezik, komunikaciju, konverzaciju
- Definicije softverske arhitekture: „ Softverska arhitektura je kolekcija dizajnerskih odluka koje se skupe za menjanje."

Podela stilova po kategorijama

Stilovi se mogu razmatrati sa različitog aspekta:
- Komunikacija
  - Servisno-orijentisana arhitektura (SOA)
  - Arhitektura zasnovana na porukama (Message Bus)
- Raspoređivanje

    - Klijent/Server
    - N-slojna
    - 3-slojna
- Domen
    - Dizajn vođen domenom
- Struktura
    - Arhitektura zasnovana na komponentama
    - Objektno-orijentisana
    - Slojevita arhitektura

Kombinacije stilova

- Stilovi nisu isključivi. Primeri:
    - SOA dizajn se može sastojati od servisa koji koriste slojevitu arhitekturu i objektno- stil
    - Web aplikacije sa slojevitom arhitekturom – odvaja se prezentaciona logika od poslovne logika i logike podataka – 3 ili više slojeva,
        - npr. Prezentacioni sloj komunicira sa modelom preko MVC (Model-View-Controller) obrasca
        - SOA arhitektura i message-based komunikacija
    - Desktop aplikacija
        - Klijent/server i arhitektura zasnovana na komponentama
- Mnogo faktora utiče na izbor arhitekture (mogućnosti organizacije za dizajn i implementaciju, iskustvo programera, zahtevi kupca i sl.)

Klijent server

- Dvoslojna arhitektura
- Inicijalno: UI –baza podataka
- Opštije: klijent – jedan ili više servera
- Druge varijante ovog stila:
    - Klijent – Queue – Klijent sistemi
    - P2P (Peer-to-Peer) aplikacije - razvijen iz prethodnog, klijent i server mogu da zamene uloge – decentralizovana arhitektura
    - Aplikativni serveri - softversko okruženje koje omogućuje izvršavanje serverskih aplikacija i servisa i tankih klijenata – web aplikacije

Klijent server arhitektura

- Prednosti:
    - Visok nivo sigurnosti (svi podaci na serveru gde je povećana sigurnost)
    - Centralizovan pristup podacima (izmene i pristup podacima na serveru)
    - Lakše održavanje (uloge i nadležnosti računarskog sistema su jasno podeljene između servera koji komuniciraju preko mreže)
- Nedostaci
    - Tendencija da su podaci i biznis logika na jednom mestu – serveru, može negativno uticati na proširenje sistema i skalabilnost
    - Zavisnost od centralnog servera može negativno uticati na pouzdanost sistema
    - Ovi nedostaci se mogu prevazići u višeslojnim aplikacijama

Arhitektura zasnovana na komponentama

- Dekomponuje dizajn na funkcionalne ili logičke komponente sa jasno definisanim interfejsima, metodama, događajima i atributima
- Obezbeđuje viši nivo apstrakcije od objektno-orijentisanog stila

- Ne fokusira se na komunikacione protokole i deljena stanja
- Važni koncepti:
    - Višestruko korišćenje
    - Zamenljivost komponenti
    - Mogućnost upotrebe u različitim okruženjima
    - Proširivost – komponenta može biti proširenje neke postojeće komponente
    - Kapsulacija sadržaja komponente
    - Nezavisnost komponenti
- Primeri: Component Object Model (COM), Distributed Component Object Model (DCOM); Common Object Request Broker Architecture (CORBA) i Enterprise JavaBeans (EJB)
- Korišćenje middleware-a za komunikaciju između komponenata

Prednosti korišćenja komponenti

- Lako se raspoređuju
- Smanjeni troškovi (korišćenje third-party komponenti)
- Jednostavan razvoj
- Višestruko korišćenje
- Pojednostavljenje tehničke složenosti

Dizajn vođen domenom

- Objektno orijentisan stil fokusiran na modelovanje poslovnog domena
- Definisanje objekata na osnovu entiteta u okviru poslovnog domena
- Potrebno dobro poznavanje poslovnog domena
- Kompleksan dizajn
- Visoka cena
- Prednosti: • Komunikacija
    - Proširivost (modularan i fleksibilan model)
    - Mogućnost testiranja (objekti modela poslovnog domena su slabo povezani)

Slojevita arhitektura

- Logička arhitektura – razdvajanje tipova funkcionalnosti
- Slojevi mogu biti smešteni na istom računaru (tier) ili na različitim računarima (višeslojni stilovi: N-tier i 3-tier stilova)
- Komunikacija komponenata u istom ili susednim slojevima (nekada i sa svima ispod)
- Fleksibilna i laka za održavanje

Prednosti slojevite arhitekture

- Apstrakcija
- Izolacija - olakšan razvoj, testiranje i održavanje aplikacije
- Performanse – distribuirani sistem - skalabilnost, otpornost na greške
- Višestruko korišćenje (npr. MVC – kontrolera)
- Testiranje
- Slojevi se mogu zameniti sa alternativnim implementacijama (razvijenim i u drugim kompanijama, GIS, SCADA i sl.)
- Minimizira se zavisnost između slojeva u cilju što nezavisnijih promena
- Slojevi predstavljaju dobra mesta za standardizaciju

Nedostaci slojevite arhitekture

- Kaskadne promene
- Nije adekvatna za jednostavne sisteme
- Dodatni slojevi mogu narušiti performanse

Arhitektura zasnovana na porukama

- Softverski sistem može da prima i šalje poruke koristeći jedan ili više kanala komunikacije
- Obično koristi ruter poruka ili Publish/Subscribe obrazac uz korišćenje Message Queuing-a
- Omogućuje:
  - Komunikaciju zasnovanu na porukama
  - Složenu logiku procesiranja • Modifikacija logike procesiranja
  - Integracija sa različitim okruženjima (npr. .NET i Java – upotrebom standarda)
- Uključuju:
  - Enterprise Service Bus (ESB) – obezbeđuje servise za transformaciju formata poruke
  - Internet Service Bus (ISB) – sličan ESB-u, ali sa aplikacijama na Cloud-u, umesto na enterprise mreži

Prednosti Message Bus stila

- Proširivost
- Jednostavnost
- Fleksibilnost
- Slaba povezanost
- Skalabilnost
- Jednostavnost aplikacija
- Pogodan za povezivanje sa eksternim apl.

Višeslojna (fizička) arhitektura – N-tier, 3-tier

- Logički predstavlja slojevitu arhitekturu, kod koje su slojevi (tier) na fizički odvojenim računarima
- minimalno 3 sloja: prezentacioni, poslovni i sloj podataka
- Predstavljaju proširenje komponentnog stila
- Funkcionalna dekompozicija aplikacija, servisne komponente, distribuiranost
- Svaki sloj je nezavisan od svih ostalih, osim što komunicira (obično asinhrono) sa susednim slojevima
- Karakteristična za bankarske Web aplikacije (važna bezbednost)
- Poslovni sloj – iza firewall-a

Karakteristike višeslojne arhitektura (II): Održavanje, Skalabilnost, Fleksibilnost, Raspoloživost

Kriterijumi za raslojavanje: Brzina odziva, Smanjenje opterećenja, Bezbednost, Tolerancija na greške - redundanost

Primer složene aplikacije

Objektno orijentisani stil

- Funkcionalnosti se raspodeljuju na objekte (komponente su objekti)
- Objekti komuniciraju putem interfejsa
- Interna reprezentacija objekta je skrivena od drugih objekata
- Prednosti: fleksibilni - proširivi, razumljivi,jednostavni za testiranje, višestruko korišćenje

Servisno orijentisana arhitektura (SOA)

- Funkcionalnosti se obezbeđuju skupom servisa
- Servisi koriste standardne interfejse
- Karakteristike servisa: Autonomnost, Distributivnost, Slaba povezanost (loosely coupled)

11. Tehnike za dizajniranje arhitekture.

- Definisanje ulaza:
  - slučajevi korišćenja,
  - funkcionalni zahtevi,
  - tehnološki uslovi,
  - razvojno okruženje,
  - dodatna ograničenja
- Proces dizajniranja:
  - Iterativni proces
  - Definisanje ciljeva, važnih slučajeva upotrebe i ključnih pitanja
  - Formiranje arhitekture

12.Osnovne faze dizajniranja softverske arhitekture.

Osnovne faze:
1. Identifikovanje ciljeva
2. Definisanje ključnih scenarija
3. Kreiranje izgleda aplikacije
4. Identifikacija ključnih pitanja
5. Definisanje kandidata (prototipa)

- Navedeni postupak je iterativan.
  - horizontalne analize - po slojevima (često se samo one rade) i
  - vertikalne analize – interakcije između slojeva
- Prvi kandidat je dizajn najvišeg nivoa, koji se doteruje

1. Identifikovanje ciljeva arhitekture
   - Definisati početne ciljeve arhitekture (da li se pravi prototip, testiraju potencijani putevi, nove arhitektura – duži proces)
   - Identifikovati korisnike (arhitekte, programeri, testeri, menadžeri)
   - Identifikovati ograničenja (softverska, hardverska, i sl)

   Primeri:
   - Kreiranje kompletnog dizajna aplikacije
   - Razvoj prototipa
   - Identifikacija ključnih tehničkih rizika
   - Testiranje dozvoljenih opcija
   - Vreme je bitan faktor (ograničenje)
2. Ključni scenariji
   - Definisanje slučajeva upotrebe – skupova interakcija između sistema i jednog ili više aktera (klijenata ili drugih sistema) - UML
   - Scenario šire opisuje interakcije korisnika i sistema od slučaja korišćenja - tekst
   - Ključni scenariji – skup najvažnijih scenarija
   - Ključni scenariji predstavljaju: problem – značajnu oblast, značajne slučajeve upotrebe, uticaj neke karakteristike (npr. bezbednost) na funkcionalnost
   - Arhitekturalno značajni slučajevi upotrebe:
     - kritični poslovni – važni za korisnika ili druge stejholdere,
     - visok uticaja na funkconalnost i kvalitet
       - npr. Create, Read, Update, Delete operacije su osetljive na bezbednost)
3. Izgled aplikacije

- Kako će aplikacija izgledati kada se završi
- Odrediti tip aplikacije
- Identifikovati infrastukturna ograničenja
- Identifikovati važne arhitektonske obrasce
- Odrediti relevantne tehnologije
    - Npr. Mobilne aplikacije, Silverlight, WPF, ASP.NET, WCF i sl.
- Skicirati arhitekturu aplikacije

4. Ključna pitanja
- Identifikovati kritična pitanja vezana za arhitekturu
    - Promena poslovnih zahteva (Da li se može preći na novi tip klijenta?)
    - Promena tehnologije (Da li se može migrirati na novu tehnologiju? Drugi third party servisi?)
- Atributi kvaliteta – osobine arhitekture koji utiču na ponašanje u realnom vremenu, projektovanje sistema i sl.
    - Kvalitet sistema – npr. testiranje i podrška
    - Kvalitet izvršavanja – npr. performanse, bezbednost, skalabilnost, pouzdanost
    - Kvalitet dizajna – npr. fleksibilnost, održavanje, mogućnost ponovnog korišćenja
    - Kvaliteti korisnika – upotrebljivost sistema

5. Rešenja - kandidati
- Nakon definisanja ključnih pitanja kreira se inicijalna osnovna arhitektura koja se popunjava
- Vrši se validacija na osnovu ključnih scenarija i definisanih zahteva
- Moguće je modifikovati arhitekturu više puta u više iteracija
- Razvijen je veći broj metodologija za pregled i procenu arhitekture

13. Slojevite arhitekture. Osnovne karakteristike.

Slojevite aplikacije
- Podela aplikacije po slojevima sa različitim funkcionalnostima i ulogama
- Najviši nivo apstrakcije - logička arhitektura sistema - skup povezanih komponenti grupisanih u slojeve
- Troslojna aplikacija najčešće sadrži:
    - Prezentacioni sloj
    - Poslovni sloj
    - Sloj podataka

Servisi i slojevi
- Više tipova klijenata
- Servisni sloj – različiti kanali za pristup aplikaciji
- pristup klijenata
    - Preko prezentacionog sloja
    - npr. direktno preko Appliation Facade
- Prezentacioni sloj može direktno komunicirati sa Poslovnim slojem (pogotovo ako su na istom računaru)

14. Koraci projektovanja slojevite arhitekture.

Koraci projektovanja slojevite strukture

1. Izabrati strategiju slojevitosti
2. Odrediti potrebne slojeve
3. Odlučiti kako distribuirati slojeve i komponente
4. Utvrditi da li treba spojiti slojeve
5. Odrediti pravila za interakciju između slojeva
6. Identifikovati poprečne nadležnosti
7. Definisati interfejse između slojeva
8. Izabrati strategiju raspoređivanja
9. Izabrati komunikacione protokole

15. Prezentacioni sloj. Komponente, karakteristike.

Komponente:

- Korisničkog interfejsa
- Prezentacione logike

Analizirati i definisati:

- Tip aplikacije,
- UI tehnologije,
- Projektantske obrasce (npr. MVC),
- Dizajniranje entiteta,
- Korisničke zahteve.

Model-View-Controller (MVC obrazac)

- Podela UI interakcije na tri izvojene uloge: View,Controller,Model
- Dve podele:
  - View – Model
    - prikaz različitih modela, različiti prikazi istog modela (Web, debeli klijent, više vrsta prikaza)
    - Prezentacija zavisi od modela, ali model ne zavisi od prezentacije
    - Subscribe/notify protokol
  - Controller – Model

Uticaj na projektovanje prezentacionog sloja: Keširanje , Komunikacija, Kompozicija, Rukovanje izuzecima, Navigacija, Korisničko iskustvo,Korisnički interfejs, Validacija

Koraci u dizajniranju prezentacionog sloja

- Identifikovati tip klijenta
- Izabrati tehnologiju
- Dizajnirati korisnički interfejs
- Odrediti strategiju validacije podataka
- Odrediti poslovnu logiku
  - UI validacije
  - Komponente poslovnog procesa
  - Model domena - da li je poslovna logika na drugom računaru
- Odrediti komunikacionu strategiju
  - Direktni pozivi
  - Web servisi

16. Poslovni sloj. Komponente. karakteristike.

Obično sadrži komponente:
- Application Facade
  - Obezbeđuje interfejs ka komponentama poslovne logike
- Poslovna logika
  - Workflow komponente
    - Koraci – tokovi poslovnog procesa
  - Komponente entiteta
    - npr. Customers ili Orders
    - validacija podataka

Projektantska pravila za poslovni sloj
- Da li je potrebno izdvojiti poslovni sloj?
- Identifikovati nadležnosti i korisnike poslovnog sloja.
- Ne mešati različite vrste komponenti.
- Smanjiti komunikaciju ka udaljenom poslovnom sloju
- Izbegavati čvrstu povezanost između slojeva.

Posebni projektantski zahtevi: Autentifikacija ,Autorizacija,Keširanje, Povezanost i kohezija, Rukovanje izuzecima, Logovanje, Validacija

Koraci u dizajniranju poslovnog sloja
- Kreiranje dizajna visokog nivoa
- Projektovanje poslovnih komponenti
- Projektovanje komponente poslovnih entiteta
- Projektovanje toka poslovnog procesa

17. Sloj podataka. Komponente, karakteristike.

Dizajn sloja podataka
- Najčešće komponente:
- Pristup podacima
  - Centralizacija pristupa podacima
  - Logika pristupa podacima – koristi pomoćne komponente
- Servisni agenti
  - Upravljaju komunikacijom sa određenim servisima
  - Izoluju različite zahteve, obezbeđuju npr. keširanje, mapiranje podataka

Projektantske odluke u sloju podataka
- Izabrati odgovarajuću tehnologiju za pristup podacima
- Koristiti apstrakciju (interfejse) za pristup sloju podataka
- Kapsulirati funkcionalnost pristupa podacima
- Odlučiti kako mapirati entitete aplikacije u strukture izvora podataka
- Izabrati način ažuriranja podataka
- Odlučiti kako će se upravljati konekcijama
- Utvrditi kako će se rukovati izuzecima

Uzeti u obzir bezbednosne rizike

Posebni projektantski zahtevi
- Grupisanje komandi nad bazom podataka
- Binary Large Objects (BLOBs)
- Povezivanje – konekcije

- Format podataka
- Rukovanje izuzecima
- Mapiranje objekata
- Upiti
- Procedure za smeštanje podataka (Stored Procedures)
- Transakcije
- Validacija
- XML

18. Servisni sloj. Komponente, karakteristike.

Projektovanje servisnog sloja Obično sadrži:

- Interfejse servisa
- Tipove poruka

Projektantska razmatranja u servisnom sloju

- Projektovanje servisa sa aspekta aplikacije, a ne komponente
- Dizajnirati proširive servise i data contract bez razmatranja ko su klijenti
- Dizajniranje samo za service contract
- Koristiti standardne elemente
- Projektovati sa pretpostavkom dobijanja nevalidnih zahteva
- Servis treba da može da detektuje i ponovi poruku (idempotencija)
- Servis treba da upravlja redosledom poruka (komutativnost)

Projektantski zahtevi servisnog sloja: Autentifikacija, Autorizacija, Komunikacija,Rukovanje izuzecima, Kanali poruka, Konstruisanje poruka, Endpoint poruke, Zaštita poruke, Rutiranje poruke, Interfejs servisa, Validacija

19. Karakteristike loše dizajniranog softvera.

Komponente slojevitih aplikacija

- Prezentacioni sloj
  - Komponente korisničkog interfejsa
  - Komponente prezentacione logike
    - Prezenter, kontroler, prezentacioni model, model prikaza
    - Komponente entiteti za prezentaciju
- Servisni sloj
  - Interfejs servisa
  - Tipovi poruka
- Poslovni sloj
  - Fasada aplikacije
  - Komponente biznis logike
    - Poslovnog procesa, entiteta (K)
- Sloj podataka
  - Pristupa podacima
  - Servisni agenti
- Poprečne komponente
  - Komponente za bezbednost, komunikaciju, zadatke operacionog menadžmenta (logovanje, rukovanje izuzecima, performance counter, konfiguracija)

Karakteristike loše dizajniranog softvera

- Krutost
- Krhkost
- Nemogućnost prenošenja komponenti
- Viskoznost
- Nepotrebna složenost
- Nepotrebno ponavljanje koda
- Teško razumevanje koda

20. Smernice prilikom dizajniranja komponenti.

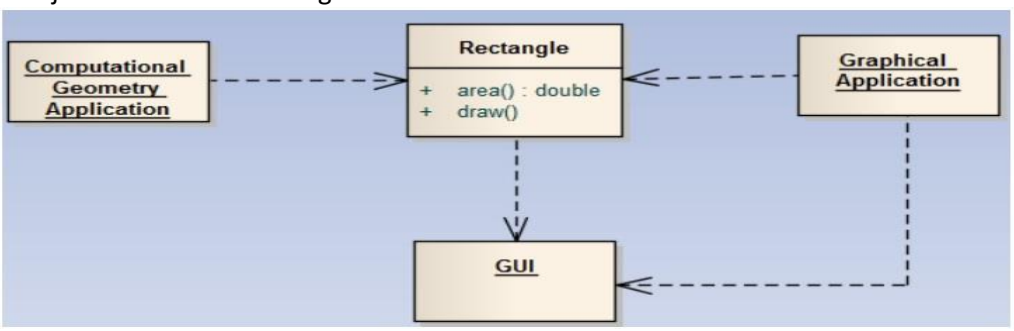
- Primeniti SOLID principe
  - S – Single Responsibility Principle
  - O – Open Closed Principle
  - L – Liskov Substitution Principle
  - I – Interface Segregation Principle
  - D – Dependency Inversion Principle
- Projektovati komponente sa velikom kohezijom.
- Na komponente ne utiču detalji implementacije druge komponente.
- Razumeti kako će komponente komunicirati.
- Odvojiti kod poprečnih funkcionalnosti (bezbednost, komunikaciju, operacioni menadžment-logovanje) od logike aplikacije.
- Primeniti ključne principe arhitekturalnog stila baziranog na komponentama.

21. Single Responsibility princip (SRP).

- Opisali su ga Tom DeMarco[1979] i Meilir Page-Jones [1988] i nazivaju ga kohezija
- Jedna klasa treba da ima samo jedan razlog za promenu (samo jednu odgovornost)
- Zbog čega je to bitno? Ako klasa ima više odgovornosti, tada ima više od jednog razloga za njenu promenu.
- Promena jedne odgovornosti može narušiti ili onemogućiti sposobnost klase da ispuni drugu odgovornost

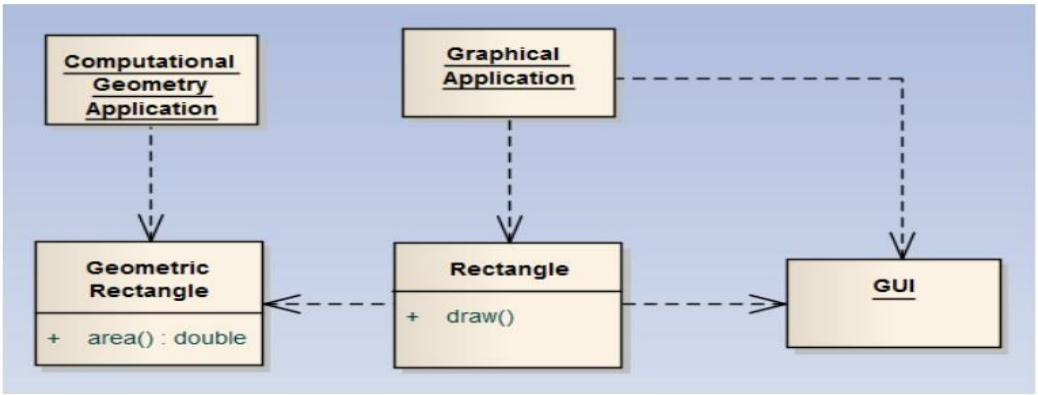
Primer 1

- Dve aplikacije koriste klasu Rectangle



- U suprotnosti sa SRP principom

Bolje rešenje

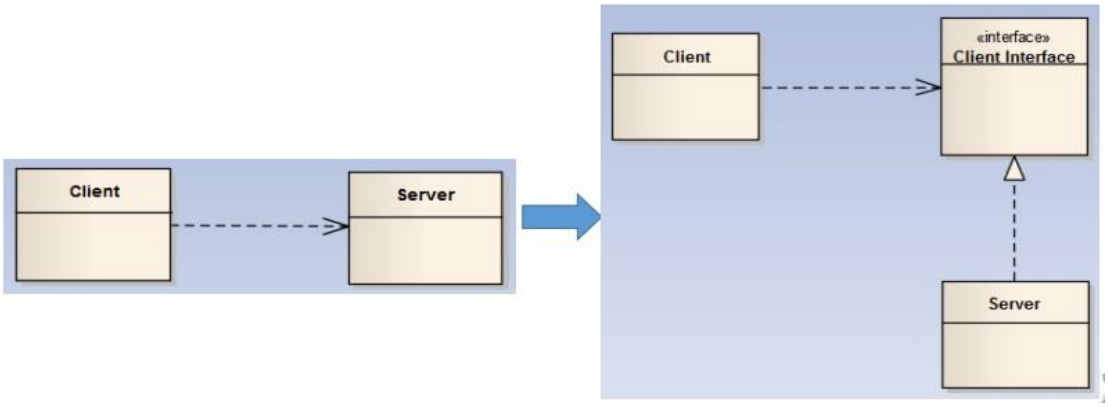


22. Open/Close princip (OCP).

Softverski entiteti (klase, moduli, funkcije,...) treba da budu otvoreni za proširenja, a zatvoreni za modifikacije.

• Apstrakcija

• Primer



23. Liskov Substitution princip (LSP).

Podtipovi moraju biti zamenljivi sa njihovim baznim tipovima

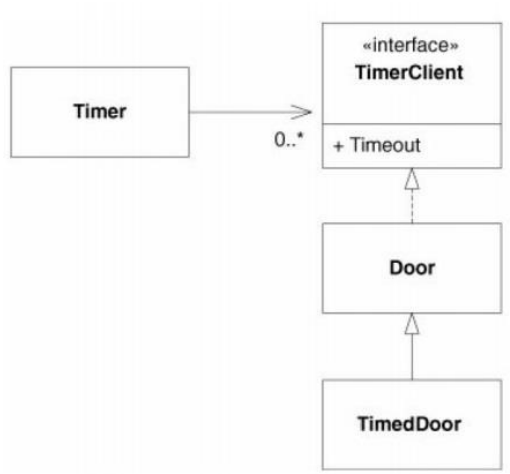
- Barbara Liskov [1988] 1 : "Ako za svaki objekat O1 klase S postoji objekat O2 klase T takav da se proizvoljni program P definisan nad klasom T ponaša isto kada se O2 zameni sa O1 , tada je S izvedena klasa iz T."

• Narušavanje LSP često uzrokuje narušavanje OCP

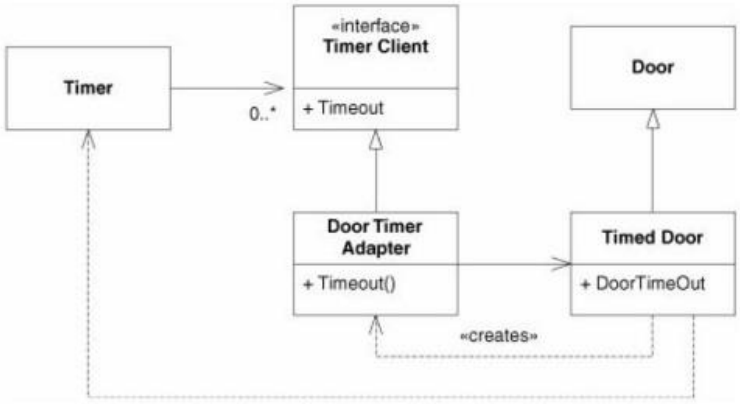
24. Interface Segregation princip.

Klijenti ne treba da zavise od metoda koje ne koriste.

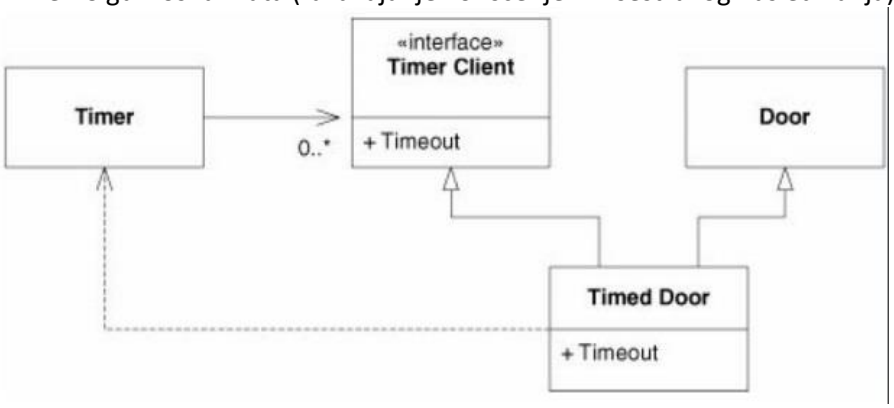
Primer: Sigurnosna vrata - Problem: Door zavisi od TimerClient! (nemaju sva vrata Timer)



Primer: Sigurnosna vrata (razdvajanje korišćenjem delegiranja)



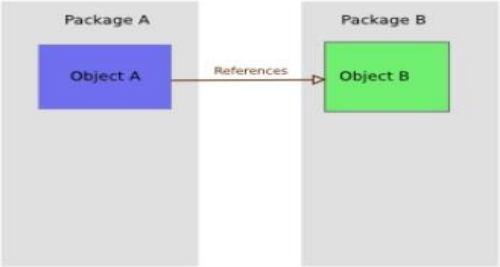
Primer: Sigurnosna vrata (razdvajanje korišćenjem višestrukog nasleđivanja)

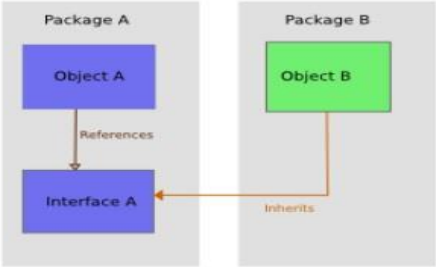


25.Dependency-Inversion princip.

A. Moduli višeg nivoa ne treba da zavise od modula nižeg nivoa. I jedni i drugi bi trebalo da zavise od apstrakcije.

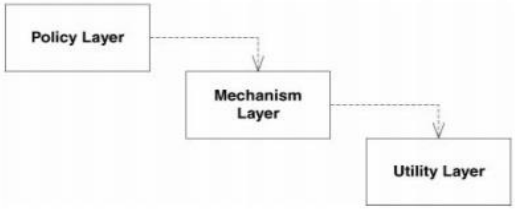
B. Apstrakcija ne bi trebalo da zavisi od detalja. Detalji bi trebalo da zavise od apstrakcije

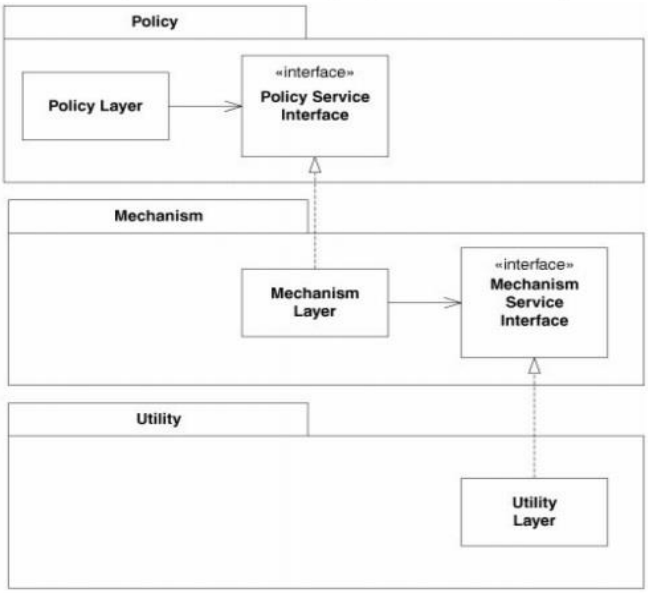




Primer

- Zavisnost je tranzitivna





- Težnja: Zavisnost od apstrakcija

26. Projektni obrazci.

Alexanderov pristup

- Tražiti zajedničke osobine u okviru problema koji se rešava
- Treba posmatrati različite građevine koje rešavaju isti problem (npr. dva trema)
- Utvrdio je da postoje sličnosti (osobine) kod kvalitetnih projekata i nazvao ih obrasci
- Definisao je obrazac kao „opšte rešenje problema“: „Svaki obrazac opisuje problem koji se stalno pojavljuje u našem okruženju i zatim opisuje srž rešenja tog problema, tako da to rešenje možete koristiti milion puta na različit način.“
- Opis obrasca sadrži 4 elementa: ime, svrhu, opis rešenja, i ograničenja i sile koje se moraju uzeti u obzir da bi se dobilo dobro rešenje.

Prelazak na softverske obrasce

- Prilagođavanje Alexander-ovog rada razvoju softvera – „Velika četvorka“ - Gamma, E., Helm, R., Johnson, R., Vlissides, J., Design Patterns: Elements of Reusable Object-Oriented Software, Reading, Mass.: Addison-Wesley, 1995.
    - Primena ideje projektnih obrazaca na programiranje
    - Opis strukture koja se koristi za opisivanje projektnih obrazaca
    - Navođenje 23 obrasca
    - Ustanovljavanje projektnih strategija i pristupa zasnovanih na projektnim obrascima
- Oni su prepoznali obrasce u postojećim, kvalitetnim, softverskim projektima
- Projektni obrasci u softverskim projektima se često nazivaju softverski obrasci

Razlozi za korišćenje softverskih obrazaca

- Ponovno korišćenje (dobrih) rešenja

- Uspostavljanje komunikacije – opšta terminologija
- Posmatranje problema sa višeg nivoa – projektovanja
- Povećana fleksibilnost koda
- Razumevanje i primena osnovnih principa objektno orijentisanog projektovanja
    - Upotreba interfejsa
    - Upotreba kompozicije radije nego nasleđivanje
    - Kapsuliranje onoga što se razlikuje

Poređenje tradicionalnog i novog pristupa (Objekti, Kapsuliranje, Apstrakne klase)

Objekti

- Tradicionalni pristup: objekti su podaci sa metodama
- Konceptualni pristup: objekti su entiteti sa odgovornostima(određuju ponašanje objekta)
- Konceptualni pristup omogućuje izradu softvera iz dva koraka:
    - Napraviti početni projekat bez udubljivanja u detalje
    - Realizovati taj projekat
- Odgovornosti pomažu da se kreira javni interfejs objekta
- Primer: objekat Shape (oblik) ima odgovornosti:
    - Da zna gde se nalazi
    - Da može da se prikaže na ekranu
    - Da može da se ukloni sa ekrana
- Potrebne metode su: getLocation, drawShape, unDrawShape
- Nema naznaka o njegovom sadržaju – sam je odgovoran za svoje podatke

Kapsuliranje

- Bilo koja vrsta skrivanja – podataka, ali i realizacija, izvedenih klasa i dr.
- Primer sa slike
- Kapsuliranje podataka (podaci u klasama Point, Line, Square, Circle)
- Kapsuliranje metoda (setLocation u Circle)
- Kapsuliranje potklasa (klijenti klase Shape ne vide potomke) Kapsuliranje ostalih objekata (samo Circle zna za XXCircle)
- Apstraktna klasa koja se ponaša polimorfno – njen klijent ne zna sa kojom vrstom potklase radi – interfejs u obrascu Adapter skriva prilagođeni objekat
- Prednosti novog pristupa – lakše održavanje softvera
    - kapsulirajući slojevi postaju interfejsi prema kojima se projektuje – dodavanjem novih oblika (Shape) klijenti se ne menjaju
- Nasleđivanje kao koncept nasuprot nasleđivanju za ponovnu upotrebu (projektuje se prema interfejsu - Shape)

Korišćenje apstraktnih klasa

- Apstraktne klase predstavljaju koncept spajanja izvedenih klasa – taj koncept se naziva srodnost (commonality)
- Razlike definisane unutar srodnosti postaju izvedene klase
- Interfejs apstraktne klase – upravlja svim odgovornostima klase

Projektovanje pomoću obrazaca

- Razumeti kontekst problema i iskoristiti obrasce za njegovo rešavanje
- „Projektovanje je često proces sinteze, spajanja i kombinovanja elemenata“ – Alexander
- Nezavisno konstruisanje delova i njihovo naknadno sklapanje nije elegantno rešenje
- Kvalitetan projekat je izgrađen sagledavanjem globalne slike problema i formiranjem delova koji se menjaju u skladu sa njegovim mestom u okviru šire celine

- Pristup projektovanju – jednostavniji zahtevi -> usložnjavanje
- Svaki obrazac je operator koji diferencira prostor celine – unošenje složenijih informacija u postavku problema – definišemo novi obrazac
- Obrasci koje Alexander opisuje definišu odnos entieta u oblasti problema

Koraci pri projekovanju

- Konceptualno definisati problem i cilj koji treba postići
- Odrediti obrasce (obrasci treba da definišu odnose između entiteta)
- Početi od obrazaca koji definišu kontekst za ostale obrasce
- Primeniti kontekstne obrasce
  - Odrediti najviši obrazac koji predstavlja kontekst za ostale
  - Primeniti taj obrazac na projekat najvišeg konceptualnog nivoa
  - Odrediti dodatne (nove) obrasce
  - Primeniti ostale kontekstne obrasce (jedan po jedan)
- Ponoviti postupak sa ostalim obrascima i definisati nove obrasce
- Usavršiti i realizovati projekat

27. Softverski obrazac – Unikat.

- Namena: Obezbeđuje da klasa ima samo jednu instancu i omogućuje joj globalni pristup.“ (V4)
- Problem: Nekoliko različitih klijentskih objekata koristi istu stvar i hoćete da obezbedite samo jednu instancu te stvari
- Rešenje: Klasa je odgovorna za postojanje svoje jedne instance.
- Učesnici: Klijent pravi instancu klase tipa Singleton preko metode getInstance
- Posledice: Klijent ne mora da vodi računa o postojanju instance

Realizacija Singleton-a

- Dodaje se privatan član instance klasi (na početku = null)
- Dodaje se javna metoda koja instancira objekat klase ako je vrednost člana nill, i vraća vrednost tog člana
- Postavlja se konstruktor na privatni ili zaštićen zbog zabrane neposrednog instanciranja objekta

Realizacija Singleton-a – više niti

- Problem istovremenog pristupa metodi getInstance
- Problemi zauzeća i curenja memorije (C++)
- Problem se može rešiti duplom proverom da li objekat već postoji (Double-Checked Locking)

28. Softverski obrazac – Komanda.

- Drugo ime – akcija, transakcija
- Namena - enkapsulira zahteve ka servisu u objekte i može se koristiti za:
  - vraćanje na stanje pre izvršavanja zahteva - undo,
  - izvršavanje zahteva preko redova čekanja,
  - slanje zahteva različitim servisima,
  - pravljenje dnevnika zahteva (logging) i sl.
- Motivacija:
  - Servis može trigerovati više različitih operacija na isti način.
  - Primer: korisnički interfejs sa objektima (dugmad, meni) – izvršavaju zahtev na osnovu ulaza korisnika
    - komplet alata GUI-ja na zna ništa o primaocu zahteva,
    - aplikacija koja koristi alate zna šta treba da uradi sa zahtevima,

• Zahtevi se pretvaraju u objekte – obrazac koristi apstraktnu klasu Komanda, npr. sa operacijom Execute – svi zahtevi su predstavljeni potklasama ove klase

Primer: Zajedničke GUI komande

• dugme Cut i Edit/Cut meni komanda aktiviraju isto ponašanje
• Ne treba duplirati kod za implementaciju zajedničkih komandi

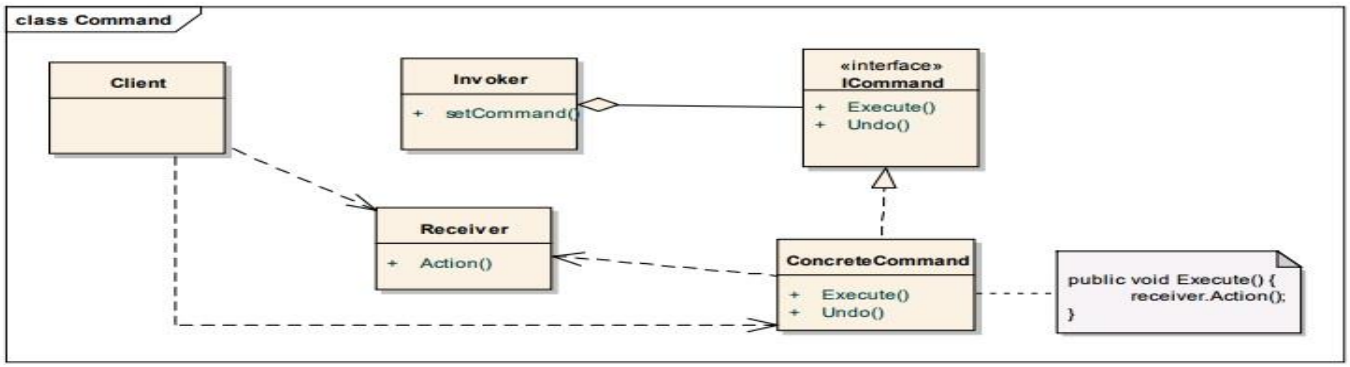
Primer: Daljinski upravljač – klase proizvođača uređaja

• Skup klasa različitih proizvođača
• Metode: • On() • Off() • setTemperature() • setVolume() • setDirection()
• Očekuje se proširenje sa klasama drugih proizvođača
• Daljinski upravljač prepoznaje samo generičke zahteve
• Kako dizajnirati akcije „Uključi svetlo“, „otvori garažu“
• REŠENJE: Obrazac Komanda

Primena obrasca Komanda

• Kada se objekti žele parametrizovati akcijom koju treba da izvrše
• Kada se zahtevi izvršavaju poređani u različito, zahtevano vreme
• Kada se želi omogućiti poništavanje akcija
• Za evidenciju promena – oporavak u slučaju pada sistema
• Za strukturiranje sistema oko operacija višeg nivoa - transakcije Obrazac ponašanja Komanda

Struktura obrasca



Učesnici

• Komanda - Definiše interfejs za izvršavanje operacija
• KonkretnaKomanda - Definiše vezu između klase Primalac i neke akcije
  - Implementira operaciju Execute() – aktivira operacije primaoca da bi izvršio zahtev
• Klijent - Pravi objekat KonkretnaKomanda i određuje primaoca
• Aktivator - Traži od komande da izvrši zahtev
  - Čuva objekat KonkretnaKomanda
• Primalac - Zna kako da izvrši operaciju kojom se ispunjava zahtev

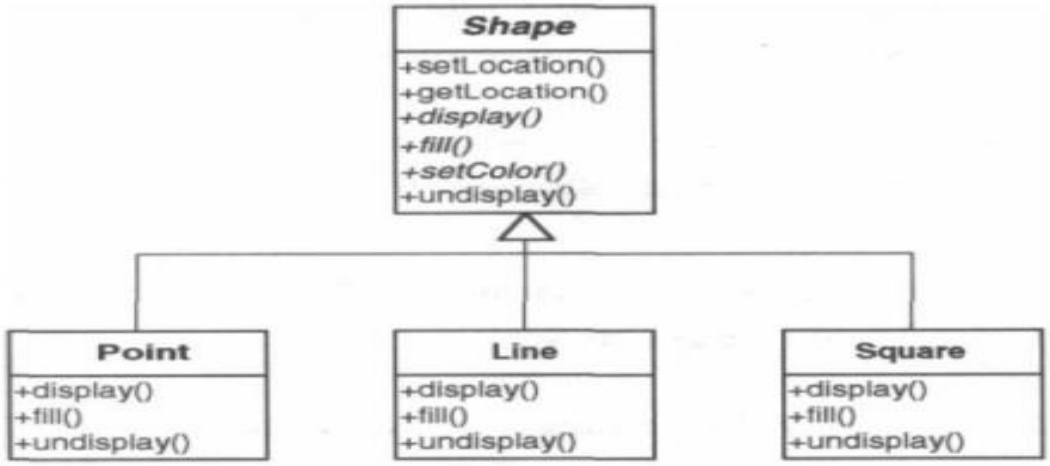
Implementacija

• Podrška za poništavanje i ponavljanje izvršavanja zahteva
• Izbegavanje akumuliranja greške u procesu poništavanja
• Moguće je sakupiti više komandi u jednu složenu – MakroKomanda
• Lako se dodaju nove komande
• U skladu sa Open-Closed principom

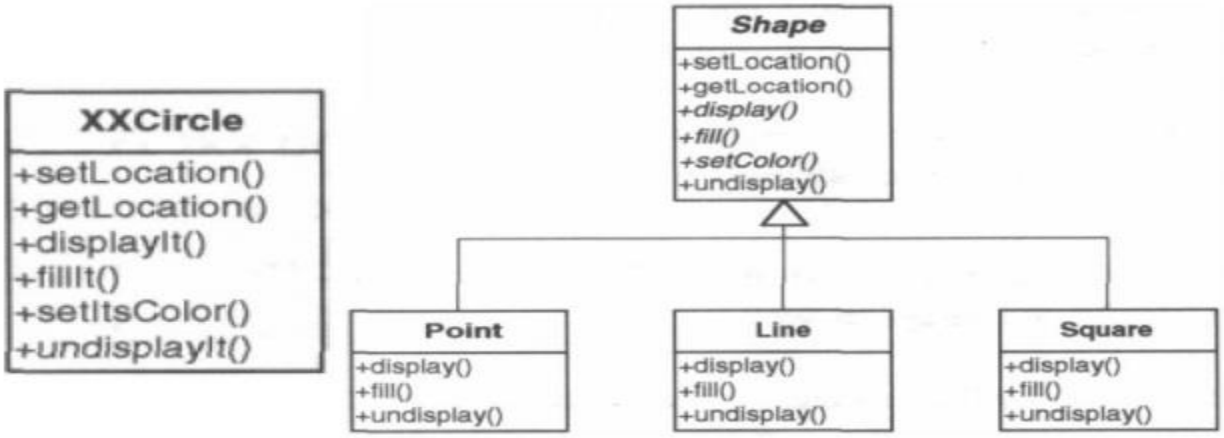
29. Softverski obrazac – Adapter.

• Pretvara interfejs klase u drugi interfejs koji klijent očekuje.
• Adapter omogućava klasama da rade zajedno, što inače ne bi mogle zbog zbog nesaglasnosti interfejsa.
• Drugi naziv - omotač (Wrapper)
• Primenjuje se kada se za postojeći, koristan objekat, pravi novi interfejs

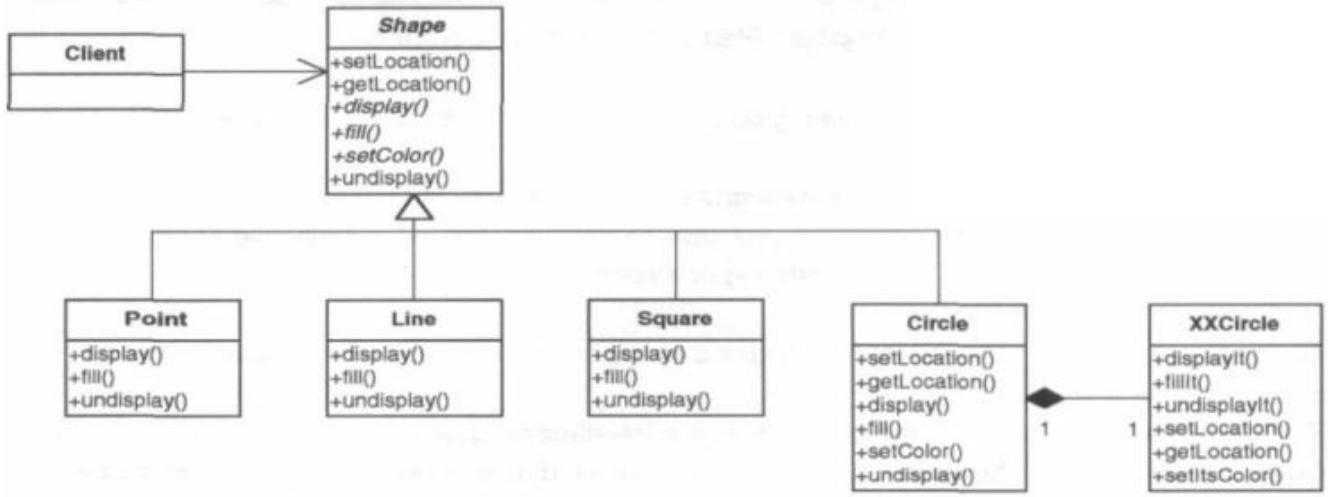
Primer



Primer - Potrebno je dodati krug ; Postoji razvijena klasa

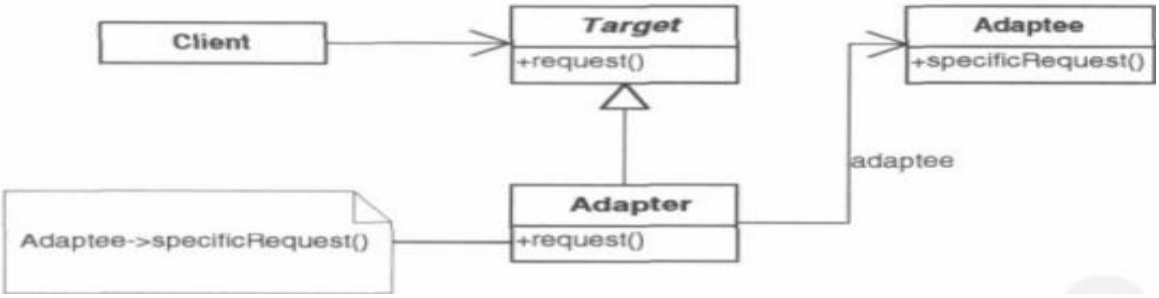


Primer – Rešenje

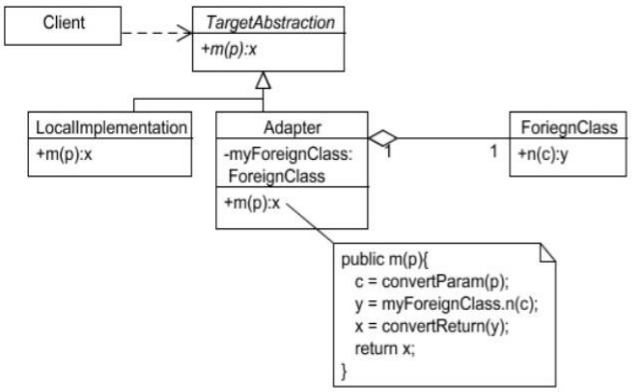


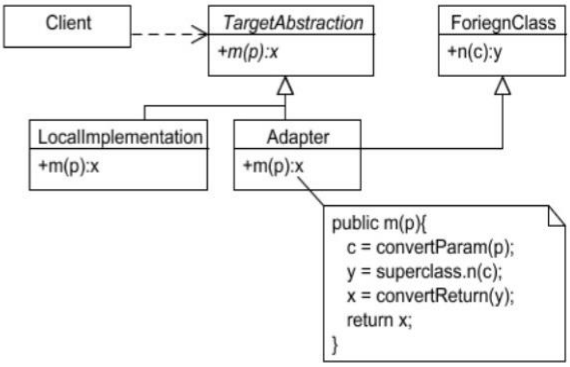
Obrazac Adapter

• Namena: Povezati postojeći objekat sa određenim interfejsom
• Problem: Sistem ima prave podatke, ali pogrešan interfejs. Obično treba izvesti neku klasu iz apstraktne klase koja se definiše ili postoji.
• Rešenje: Adapter je omotač sa odgovarajućim interfejsom.
• Ušesnici: slika
• Posledice: Obrazac Adapter – postojeći objekti se uklapaju u novu strukturu, ne ograničava ih novi interfejs
• Realizacija: Smestiti postojeću klasu u drugu



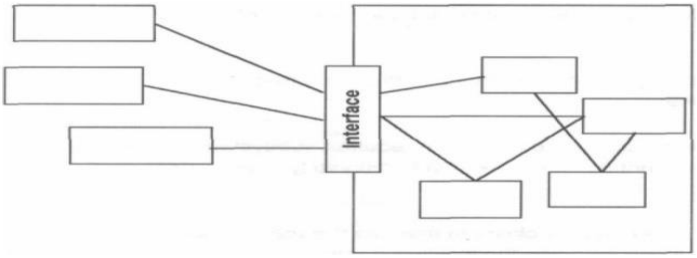
Tipovi adaptera: Objektni Klasni



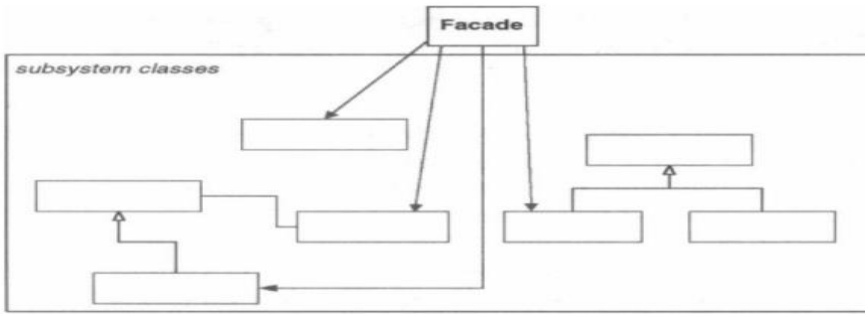


30. Softverski obrazac – Fasada.

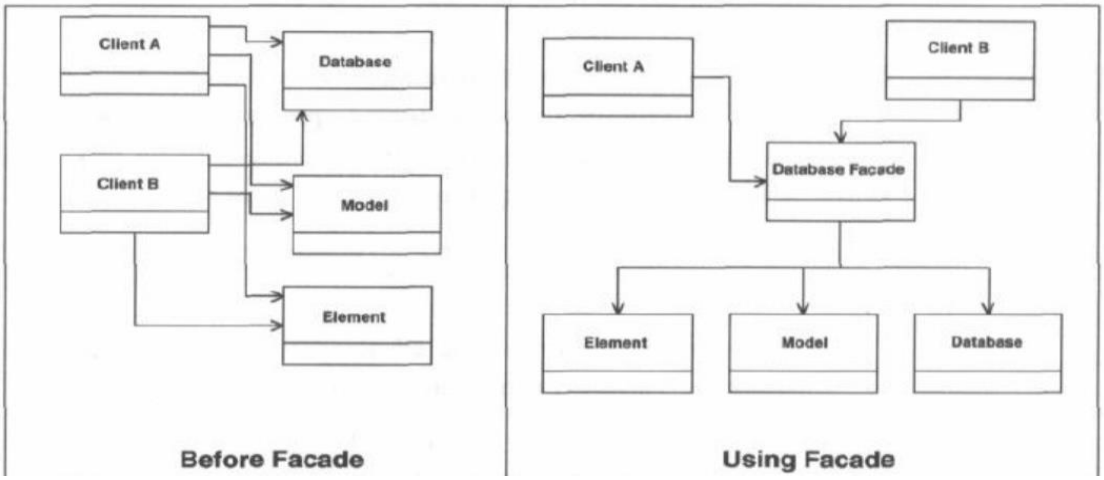
„Predstavlja jedinstven interfejs (višeg nivoa) za olakšano korišćenje interfejsa podsistema“ (V4)



Obrazac Fasada: Ključne karakteristike
• Namena: Pojednostavljenje sistema. Treba da definiše sopstveni interfejs
• Problem: Korišćenje samo dela složenog sistema ili korišćenje sistema u određene svrhe
• Rešenje: Novi interfejs namenjen klijentu postojećeg sistema
• Posledice: pojednostavljena upotreba zahtevanog sistema.
    • Slabo vezivanje između podsistema i klijenta
    • Olakšano održavanje podsistema
• Realizacija: Definiše novu klasu koja sadrži zahtevani interfejs



Primer



Kada koristiti obrazac Fasadu

- Potreban jednostavan interfejs ka podsistemu
  - Složenost sistema raste tokom razvoja
  - Potreban je samo manji deo usluga koje pruža podsistem
- Postoje brojne zavisnosti između klijenata i klasa podsistema
  - Fasada razdvaja podsistem od klijenata i drugih podsistema
- Potrebno je raslojiti podsisteme
  - Fasada je ulazna tačka novog podsistema Strukturni obrazac Fasada

Učesnici

- Fasada - Zna koje klase podsistema opslužuju zahtev klijenta
  - Prosleđuje zahteve klijenata odgovarajućim objektima podsistema
- Klase podsistema - Implementiraju funcionalnosti
  - Izvršavaju zahteve fasade
  - Ne znaju ništa o fasadi
- Klijenti - Šalju zahteve Fasadi

*Uporedne karakteristike

Aspekt	Fasada	Adapter
Postojanje klase	DA	DA
Postojanje interfejsa	NE	DA
Polimorfno ponašanje objekta	NE	Verovatno
Potreba za jednostavnijim interfejsom	DA	NE

- Fasada pojednostavljuje interfejs, dok Adapter menja interfejs u neki drug

31. Softverski obrazac –Šablonski metod.

Primer X: Recepti za instant kafu i čaj

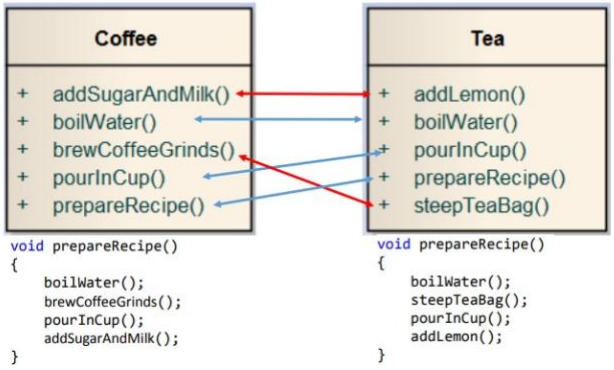
Recept za kafu

- Skuvati vodu do ključanja
- Zakuvati kafu
- Sipati kafu u šoljicu
- Dodati šećer i/ili mleko

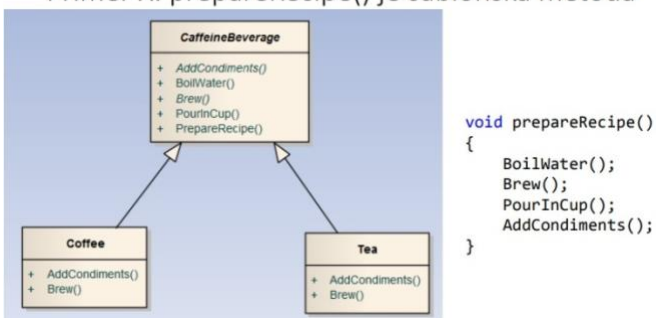
Recept za čaj

- Skuvati vodu do ključanja
- Potopiti čaj u vruću vodu
- Sipati čaju u šoljicu
- Dodati limun

Primer X: Klase



Primer X: prepareRecipe() je šablonska metoda

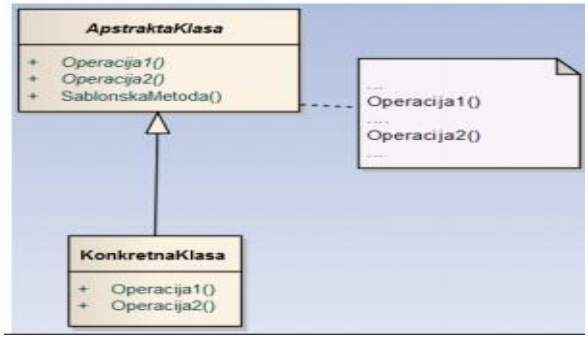


Šablonski metod - opis

- Obrazac šablonski metod definiše kostur algoritma u nekoj operaciji klase, čiji se pojedini koraci definišu u klasama naslednicama
- Definisanje invarijantnih delova koda u apstraktnoj klasi
- Invertovanje kontrole

Hollywood princip: "Don't call us, we'll call you".

- Upotreba tzv. hook operacija
  - Obično ne rade ništa ili su sa podrazumevanim ponašanjem
- Opcione su
- Mogu se proširiti (ali ne moraju) u klasama naslednicama

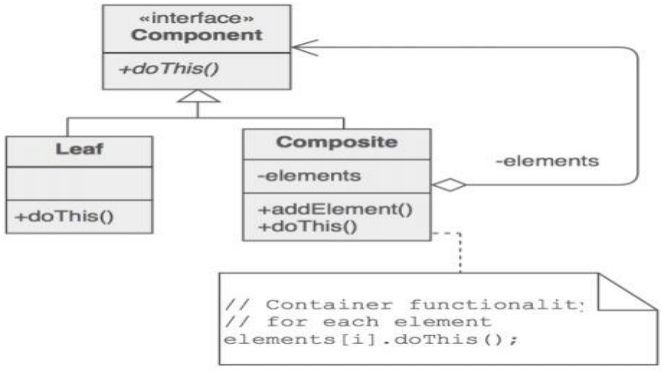


32. Softverski obrazac –Sastav.

- Namena:
  - Kompozicija (sastavljanje) objekata u strukt. u vidu stabla da bi se predstavila hijerar. delova I celina
  - Klijenti uniformno tretiranje proste i složene objekte.
  - Npr. „Direktorijumi sadrže stavke, od kojih neke mogu biti direktorijumi"
  - Upravljanje potomcima – addChild(), removeChild()
- Motivacija:
  - Editori za crtanje – korisnici mogu da grade složene dijagrame pomoću jednostavnih komponenti ili grupisanih složenih komponenti
  - Apstraktna klasa Graphic predstavlja osnovne oblike i njihove kontejnere
  - „Koristiti ovaj patern kada se jedna komponenta sastoje od komponenata koja se može smatrati složenim sastavom (nekih komponenata)"

Struktura

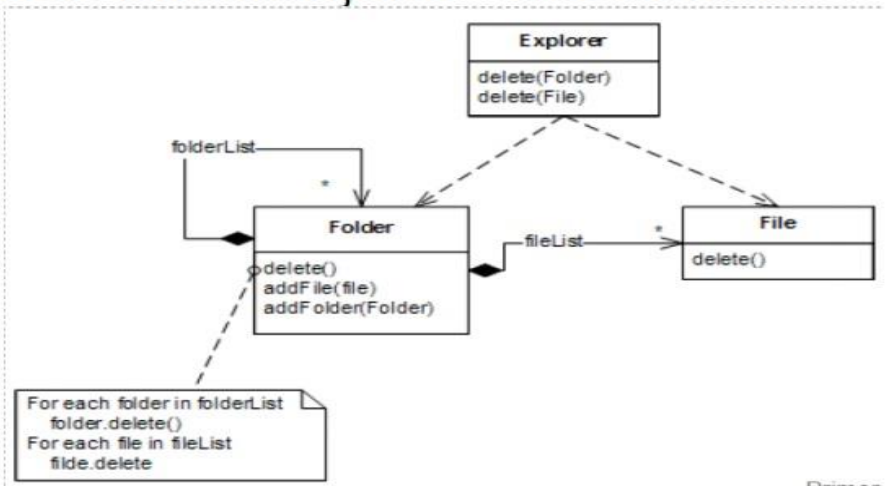
- Apstraktna klasa (Component) definiše jedinstveno ponašanja
- Composite klasa definiše ponašanje složenih komponenti (sa decom) i čuva komponente potomaka



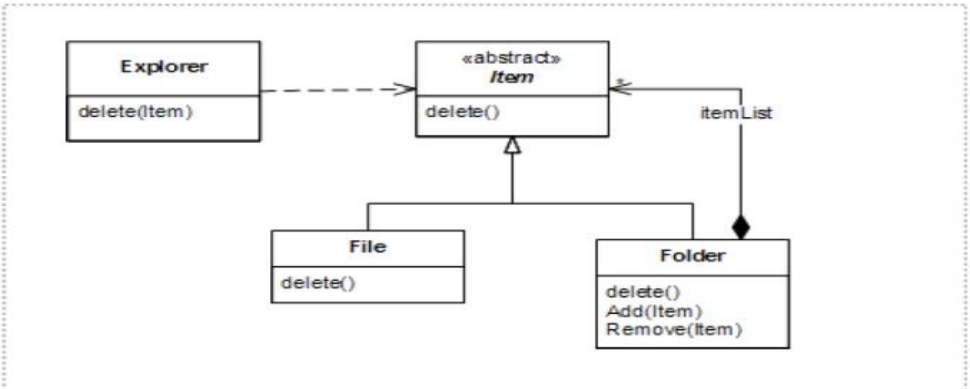
Učesnici

- Komponenta (u primeru - Graphic)
  - Deklariše interfejs za objekte
  - Implementira podrazumevano ponašanje za zajedničke metode svih klasa
  - Deklariše interfejs za pristup i upravljanje komponentama potomcima
- List (u primeru - Line, Rectangle, Text)
  - Predstavlja i definiše ponašanje prostih objekata - listova (bez potomaka)
- Sastav (u primeru - Picture)
  - Definiše ponašanje složenih komponenti (sa potomcima)
  - Čuva potomke
  - Implementira operacije nad potomcima
- Klijent
  - Manipuliše objektima u sastavu preko interfejsa Komponenta

Primer: Direktorijumi – bez obrasca



Primer: Direktorijumi – rešenje sa obrascem Sastav

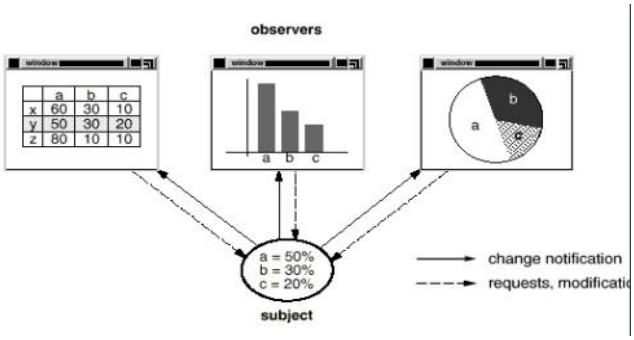


Posledice

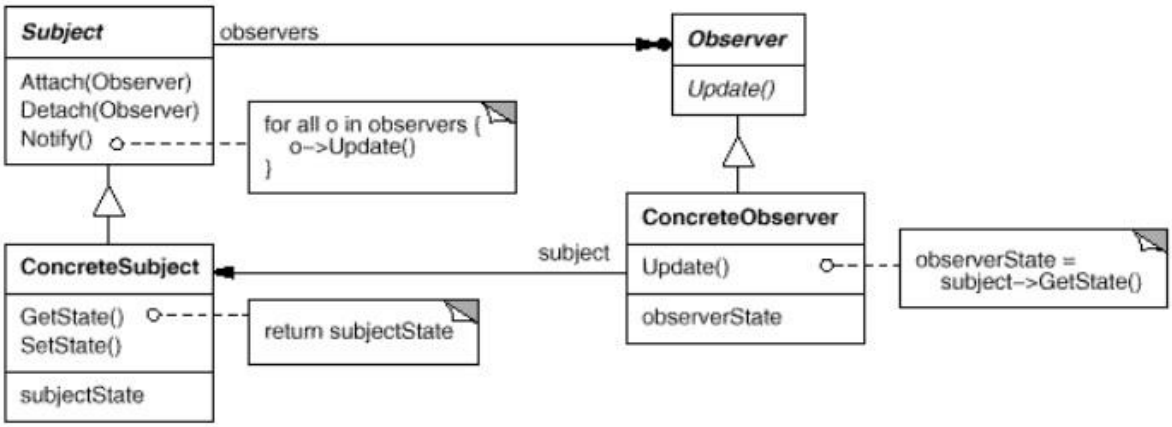
- Definiše hijerarhiju klasa (prostih i složenih)
- Pojednostavljenje korišćenja za klijenta – jednoobrazni pristup
- Olakšano dodavanje novih komponenti
- Mana: Može da dovede do previše uopštenog dizajna – ne mogu se slobodno postaviti ograničenja (uključiti samo određene komponente u sastav složene komponente)

33. Softverski obrazac – Posmatrač.

- Namena
  - Definiše zavisnost 1 prema više objekata
  - Prilikom promene stanja jednog objekta obaveštavaju se i ažuriraju svi zavisni objekti
- Ime
  - Poznat i kao Dependents (Zavisnici) ili Publish-Subscribe
- Motivacija
  - Npr. Više otvorenih prozora prikazuju ista stanja i klijenti ih ažuriraju
  - Promene u jednom objektu zahtevaju promene u drugim objektima
  - Kada se izbegava tesna povezanost između objekata – uvodi se objekat za obaveštavanje



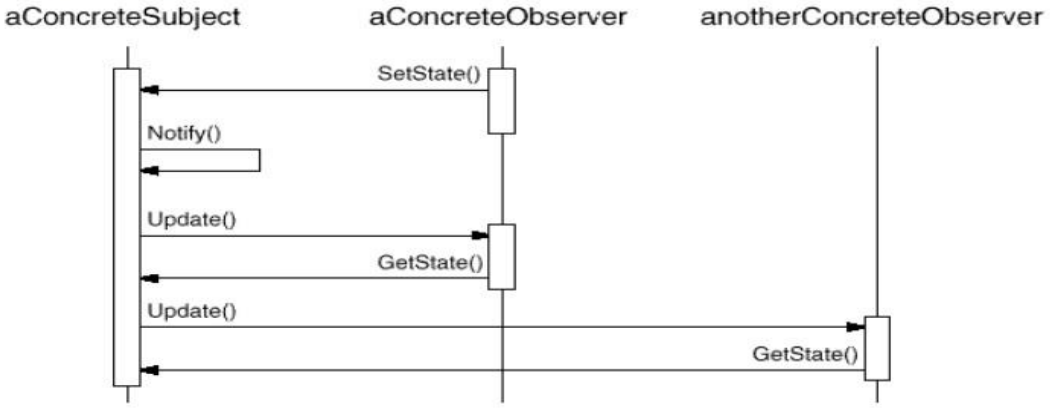
Struktura



Učesnici

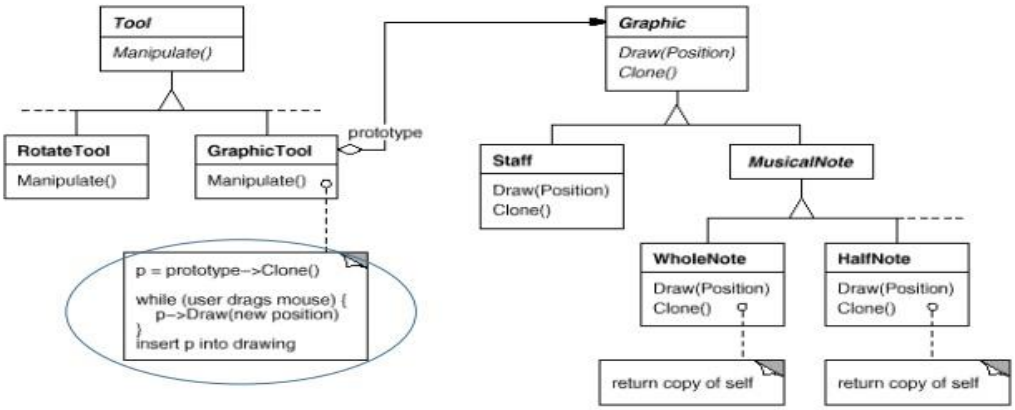
- Subjekat
  - Zna za posmatrače (proizvoljan broj) i obezbeđuje interfejs za povezivanje i raskidanje veze
- Posmatrač
  - Definiše interfejs za ažiriranje objekata koje treba obavestiti o promenama subjekta
- Konkretan subjekat
  - Čuva stanje od interesa za Konkretan posmatrač i obaveštava posmatrača kada se stanje promeni
- Konkretan posmatrač
  - Održava referencu na Konkretan subjekat, čuva stanje (konzistentno sa KS) i implementira Posmatrač-ev interfejs za ažuriranje

Interakcija



Posledice
- Apstraktno vezivanje Subjekta i Posmatrača
- Podrška za difuznu komunikaciju
- U slučaju više Posmatrača potrebno je identifikovati objekat koji je izvršio promenu – izbegavanje beskonačne petlje

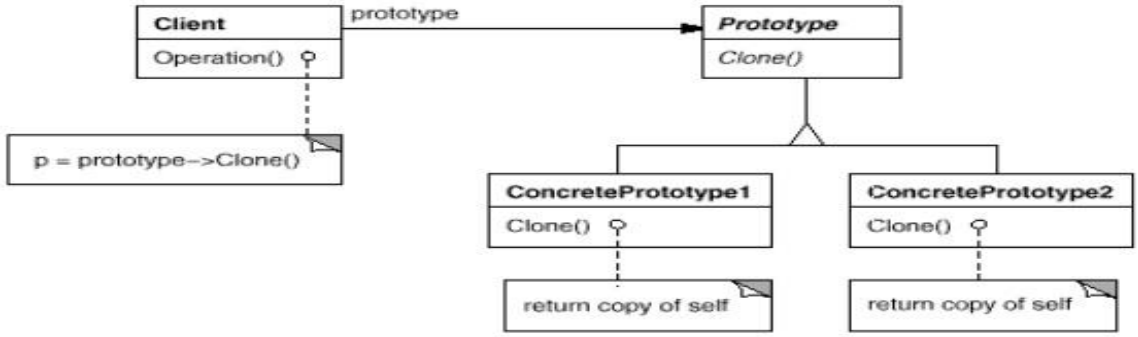
34. Softverski obrazac – Prototip.
- Namena:
  - Određivanje vrste objekata koji se prave koristeći prototipski primerak i prave nove objekte kopiranjem tog prototipa
- Motivacija:
  - Editor za muzičke partiture
    - Dodavanje muzičkih objekata u partituru – paleta alatki (biranje, premeštanje i brisanje objekata)
    - Objekti su note, pauze i notne linije
    - Radno okruženje obezbeđuje
      - Apstraktnu klasu Graphic za grafičke komponente (nota, notna linija)
      - Apstraktnu klasu Tool za za definisanje alata
      - Klasu GraphicTool za kreiranje i dodavanje grafičkih objekata u dokument
  - Note i notne linije specifične za aplikaciju, GraphicTool pripada radnom okruženju (kako GraphicTool da pravi muzičke objekte u radnom okruženju – pravi se novi objekat klase Graphic)



Primenljivost

- Kada sistem treba da bude nezavisan od načina kreiranja, sastavljanja i predstavljanja njegovih delova
- Kada se klase koje treba da se instanciraju određuju u toku izvršavanja, npr. dinamičkim učitavanjem
- Kada kreiranje objekta zahteva puno vremena, a imamo sličnu instancu klase
- Umesto kreiranja novog složenog objekta koristi se postojeći – sličan objekat
- Instanca nastala kopiranjem treba da obezbedi sopstveno kopiranje
- Podrška u C#:
    - MemberwiseClone() – „plitko kopiranje“ (engl. shallow copy) - ne kopira reference i
    - IClonable interfejs (sa Clone metodom) – pravi novu instancu sa svim vrednostima istip kao i postojeća – korisnik može da implementira izmenjenu Clone() metodu kao deep copy ili shallow copy

Struktura



Učesnici

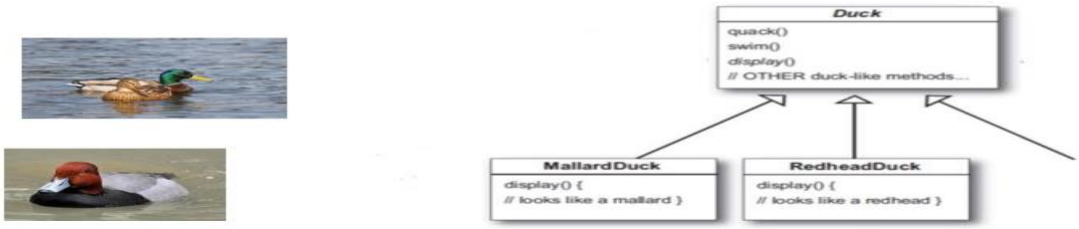
- Prototip - Deklariše interfejs za svoje kloniranje
- Konkretan prototip - Implementira operacije za svoje kloniranje
- Klijent - Pravi novi objekat - zahteva od prototipa da se klonira

Posledice

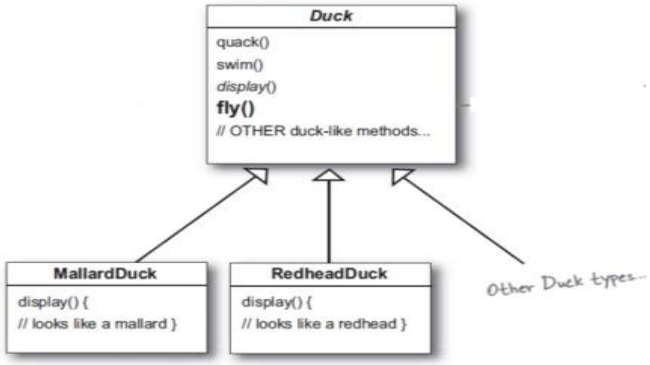
- Prednosti
    - Dodavanje i uklanjanje proizvoda u vreme izvršavanja
    - Određivanje novih objekata menjanjem vrednosti
    - Određivanje novih objekata menjanjem strukture
    - Smanjenje broja potklasa
    - Dinamičko konfigurisanje aplikacije klasama
- Mane
    - Za svaku potklasu Prototipa mora se implementirati metoda Clone() ili Copy() (npr. problem za klase koje se uzajamno referenciranja)

35. Softverski obrazac – Strategija.

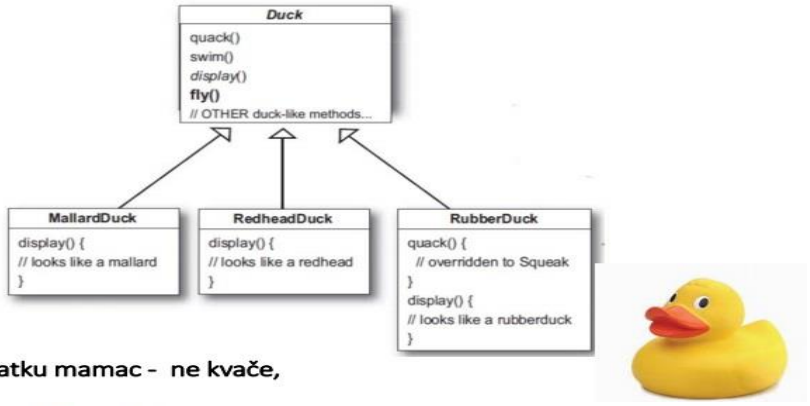
Primer: Simulaciona igrica sa patkama



Primer: dodata metoda Fly()

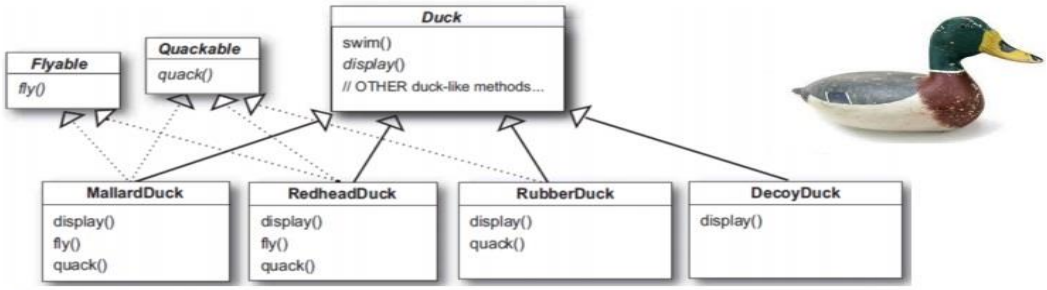


Dodata vrsta gumena patkica – ne kvače-pišti, ne leti



Treba dodati patku mamac - ne kvače, ne leti

Rešenje nasleđivanjem

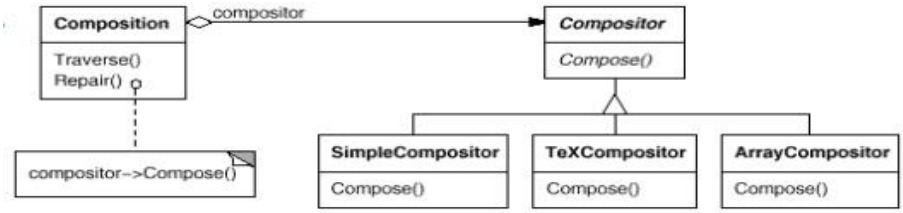


- Problem proširivosti
- Dizajnerski pristup: Identifikovati aspekte aplikacije i odvojiti ih od nepromenljivosti

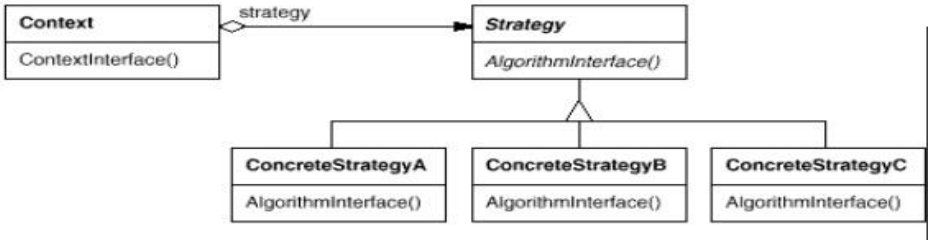
• Namena: definiše familiju algoritama, svaki kapsulira i čini ih međusobno zamenljivim (nezavisno od klijenta).

• Ime: Poznata i kao Politika (Policy)

• Motivacija - Prelom teksta na redove



Struktura



Učesnici

• Strategija - Deklariše zajednički interfejs za sve algoritme
• KonkretnaStrategija - Implementira algoritam pomoću interfejsa strategija
• Kontekst
  • Konfiguriše se pomoću objekta KonkretnaStrategija
  • Održava referencu na jedan objekat Strategija
  • Može omogućiti da Strategija pristupa podacima (predaje podatke ili sebe)
  • Prisleđuje zahteve klijenata svojoj strategiji
  • Klijenti prave i predaju Kontekst-u objekat KonkretnaStrategija i nakon toga klijenti sarađuju samo sa Kontekst-om

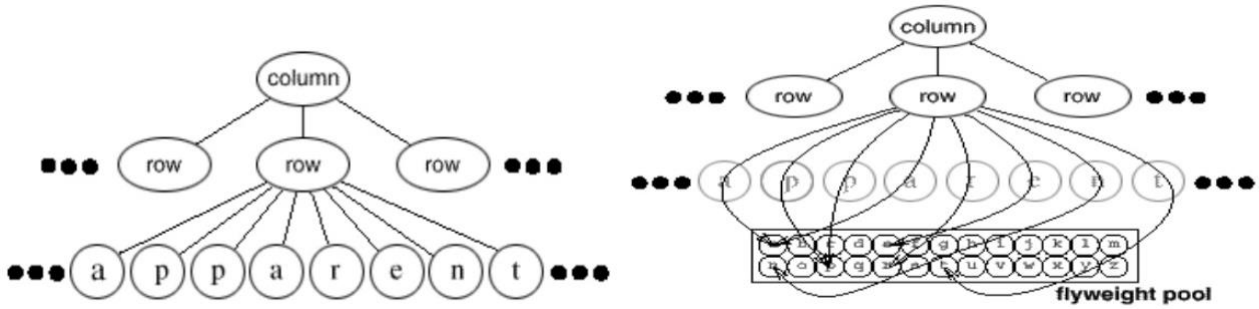
Posledice

  • Familije srodnih algoritama
  • Alternativa za potklase (Kontekst-a)
  • Eliminiše uslovne iskaze
  • Izbor implementacija
  • Mane
    • Klijenti moraju biti svesni različitih strategija
    • Dodatno opterećenje komunikacijom između Strategije i Konteksta
    • Uvećan broj objekata

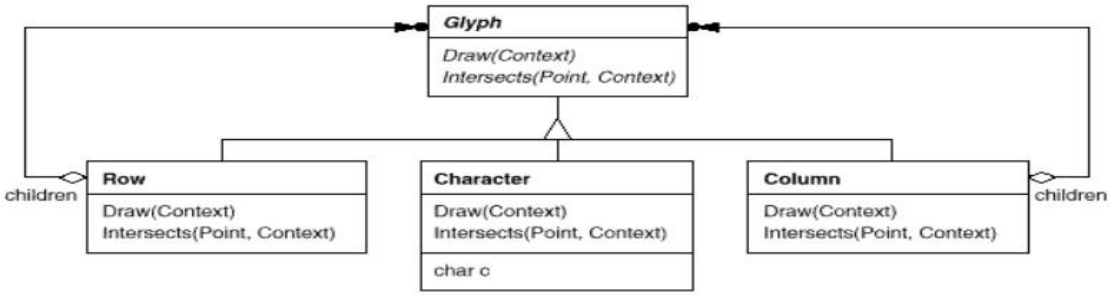
36. Softverski obrazac – Muva.

• Namena - Koristi deljenje za rukovanje velikog broja sitnijih objekata
• Motivacija
  • Editor dokumenata – uređenje i formatiranje teksta
  • Muva – deljeni objekat: Unitrašnje stanje, Spoljašnje stanje

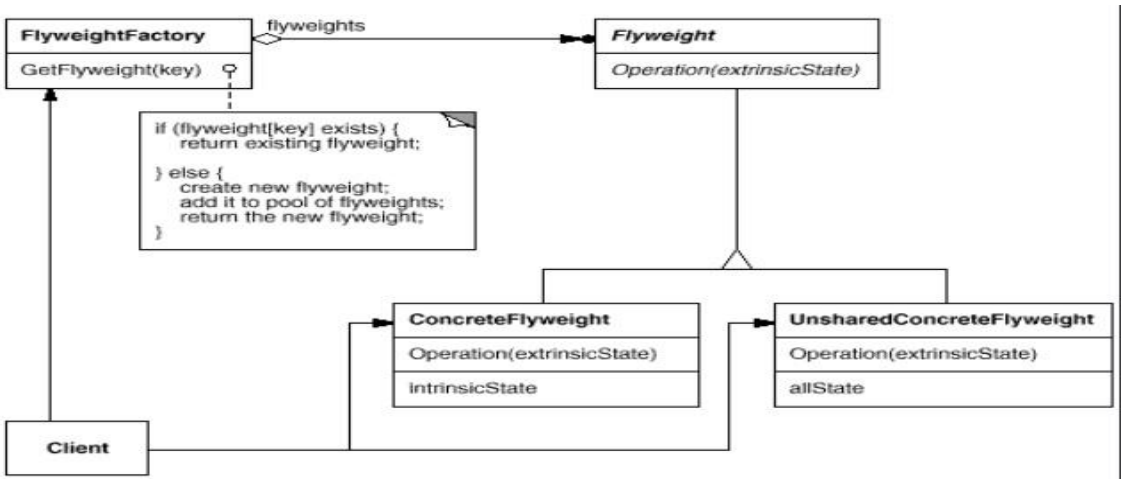
Primer: Editor teksta



Primer: Editor teksta – dijagram klasa

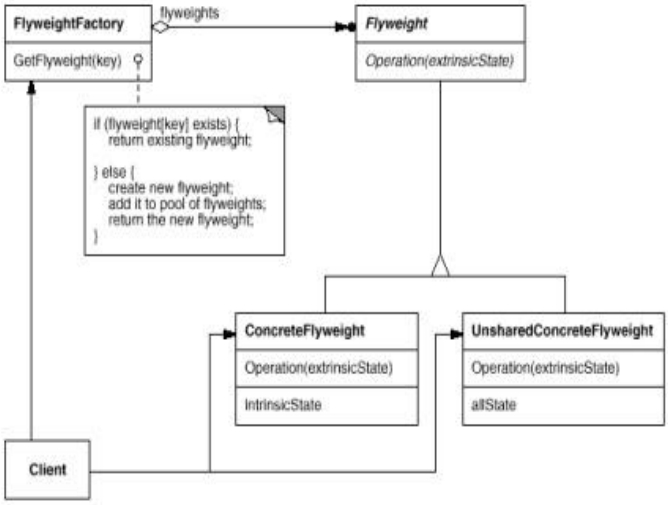


Struktura obrasca



Učesnici

- Flyweight(Glyth)
  - Deklariše interfejs za spoljašnje stanje
- ConcreteFlyweight (Character)
  - Implementira interfejs Muva
  - Definiše unutrašnje stanje – nezavisno od svog kontektsta
  - Mora biti deljiv
- UnsharedConcreteFlyweight
  - (Row, Collumn)
- FlyweightFactory
  - Pravi objekte, upravlja njima, obezbeđuje ih klijentima
- Client
  - održava referencu na objekat Muva,
  - izračunava ili čuva spoljašnje stanje



Prednosti
• Smanjuje se broj objekata - štedi memoriju.
• Objedinjuje stanje većeg broja objekata – unutrašnje stanje u jednom objektu.
• Veći deo stanja objekta može da postane spoljašnje stanje – prednost ako se spoljašnje stanje može izračunati (umesto da se čuva)
Mane
• Fliveight se koristi kada klasa ima mnogo primeraka, a svi oni mogu biti kontrolisani identično.
• Kada se implementira Muva obrazac, pojedinačne instance klase neće biti u mogućnosti da funkcionišu nezavisno od drugih instanci.

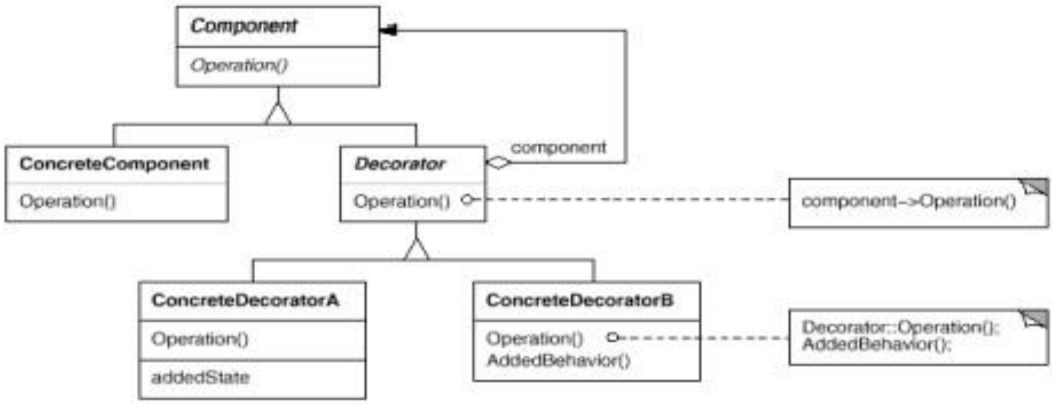
37. Softverski obrazac – Dekorater.

Primer – ponuda napitka (slajdovi)

• Dinamičko dodeljivanje objektu dodatnih odgovornosti
• Dekoratori omogućuju fleksibilno dodavanje podklasa sa proširenjem funkcionalnosti

• Namena:
  • Dinamičko pridruživanje dodatnih odgovornosti objektu
  • Fleksibilna alternativa mehanizmu nasleđivanja u cilju proširenja funkcionalnosti
• Alternativno ime
  • Uvijač (wrapper)
• Motivacija
  • Dodavanje pojedinim prozorima GUI-ja dodatnih svojstava
    • okvir, traka za pomeranje
  • Dinamički i transparentno (bez uticaja na druge objekte)
  • Odgovornosti koje mogu da se povuku

Struktura



Učesnici
  • Komponenta
    • Definiše interfejs za objekte kojima se mogu dinamički dodavati odgovornosti
  • KonkretnaKomponenta
    • Definiše objekat za koji se vezuju dodatne odgovornosti
  • Dekorater
    • Održava referencu na objekat tipa Komponenta
    • Definiše interfejs usklađen sa interfejsom Komponente
  • KonkretanDekorater
    • Dodaje odgovornost Komponenti

Karakteristike

Prednosti

- Veća fleksibilnost od nasleđivanja
- Klase bogate karakteristikama nisu visoko u hijerarhiji

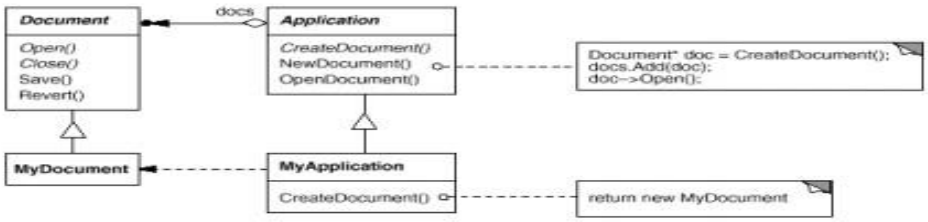
Mane

- Dekorater i komponenta nisu identični (ne treba se oslanjati na identitet korišćenog objekta)
- Mnogo malih objekata

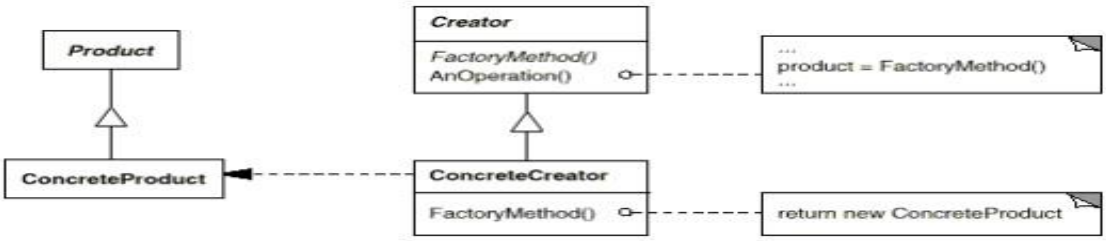
- Srodni obrasci: Adapter, Sastav i Strategija

38. Softverski obrazac – Proizvodni metod.

- Namena:
  - Odvajanje klijenta od konkretne implementacije i klase koju instancira
  - Definiše interfejs za pravljenje objekta
- Motivacija:
  - Radno okruženje koristi apstraktne klase za definisanje i održavanje odnosa među objektima



Struktura



Učesnici

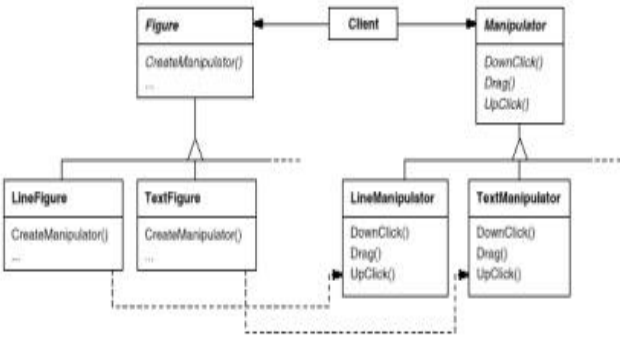
- Proizvod - Definiše interfejs objekta koji pravi proizvodni metod
- KonkretanProizvod - Implementira interfejs Proizvod
- Kreator - Deklariše proizvodni metod koji vraća objekat tipa Proizvod i može ga pozvati
- KonkretanKreator - Nadjačava proizvodni metod i vraća instancu KonkretnogProizvoda

Karakteristike

- Eliminiše potrebu za klasama specifičnim za pojedine aplikacije
- Obezbeđuju potklasama sredstvo za proširenje verzije objekta – povećana fleksibilnost
- Povezuju paralelne hijerarhije klasa

Mane:

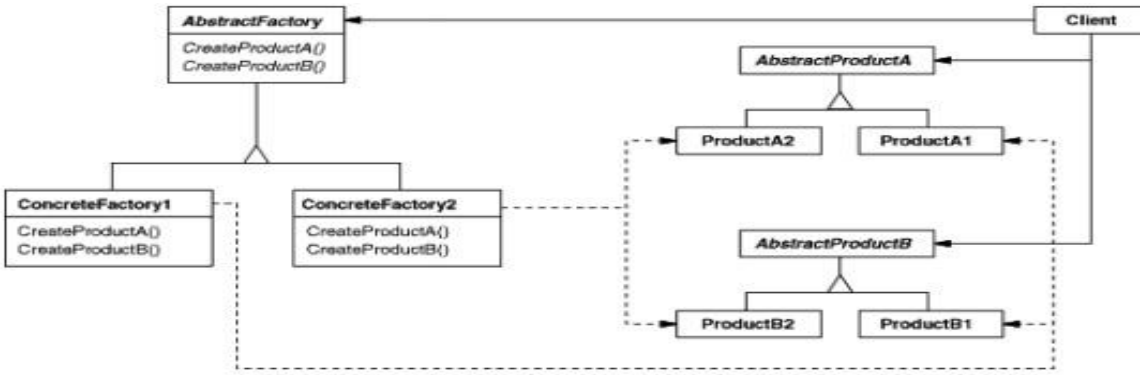
- Relativno veliki broj klasa
- Svaki Proizvod zahteva svog Kreatora



39. Softverski obrazac – Apstraktna fabrika.

- Namena:
  - predstavlja interfejs za pravljenje familija povezanih objekata (kojih klijenti nisu svesni)
  - Proširenje Factory Method obrasca
- Motivacija:
  - Alati za korisnički interfejs
  - Softver za auto servise:
    - Tojota radi sa Tojotinim Delovima, Ford sa Fordovim…

Struktura



Učesnici

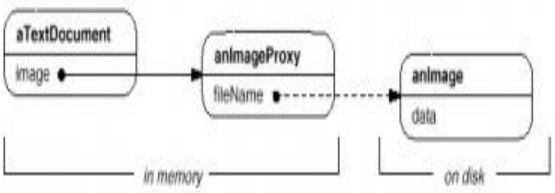
- ApstraktnaFabrika - Deklariše interfejs za operacije koje prave apstraktne objekte proizvode
- KonkretnaFabrika - Implementira operacije za pravljenje objekata KonkretnihProizvoda
- ApstraktniProizvod - Deklariše interfejs za jednu vrstu proizvoda
- KonkretanProizvod - Definiše objekat Proizvod (koji se pravi)
  - Implementira interfejs deklarisan u ApstraktnomProizvodu
- Klijent - Koristi samo interfejse deklarisane u klasama ApstraktnaFabrika i ApstraktniProizvod

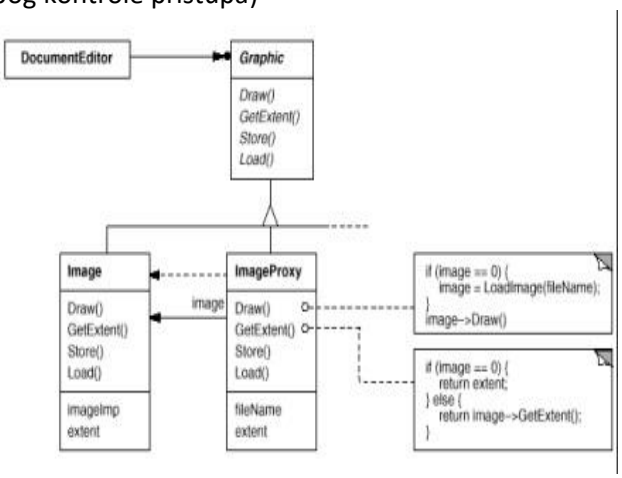
Karakteristike

- Izoluje konkretne klase
- Olakšava zamenjivanje familije proizvoda
- Unapređuje se konzistentnost proizvoda
- Mana: Teško se podržavaju novi proizvodi (zahteva se izmena interfejsa ApstraktnaFabrika)

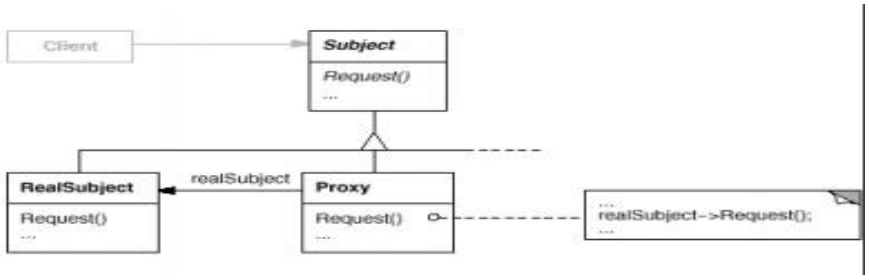
40. Softverski obrazac –Proksi.

- Namena
  - Posrednik za pristupanje drugom objektu (obično zbog kontrole pristupa)
- Motivacija
  - Kontrolisani pristup
  - Vrste:
    - Daljinski (Remote Proxy),
    - Virtuelni (Virtual Proxy),
    - Zaštitni (Protective Proxy),
    - Pametni (Smart Proxy).



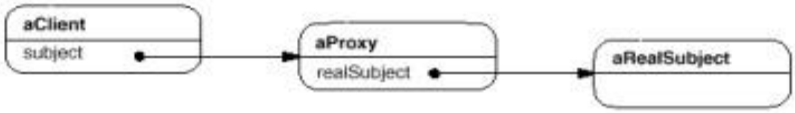


Struktura



Učesnici

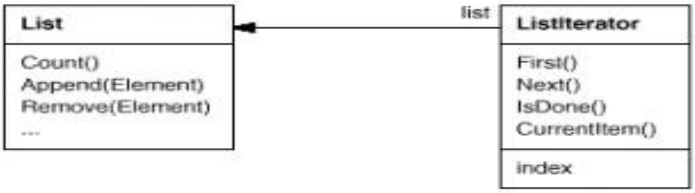
- Proksi
  - Održava referencu koja mu omogućava pristup stvarnom objektu
  - Obezbeđuje interfejs identičan interfejsu Subjekta
  - Kontroliše prava pristupa
- Subjekat
  - Definiše zajednički interfejs za Proksi i PraviSubjekat
- PraviSubjekat
  - Definiše pravi objekat koji proksi predstavlja



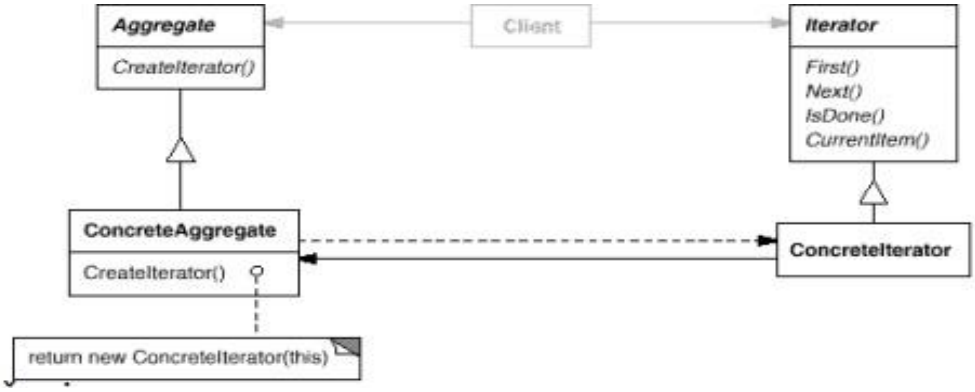
- Primer u C#: MarshalByRefObject klasa za pristup objektima iz drugog domena (remoting)

41. Softverski obrazac –Iterator.

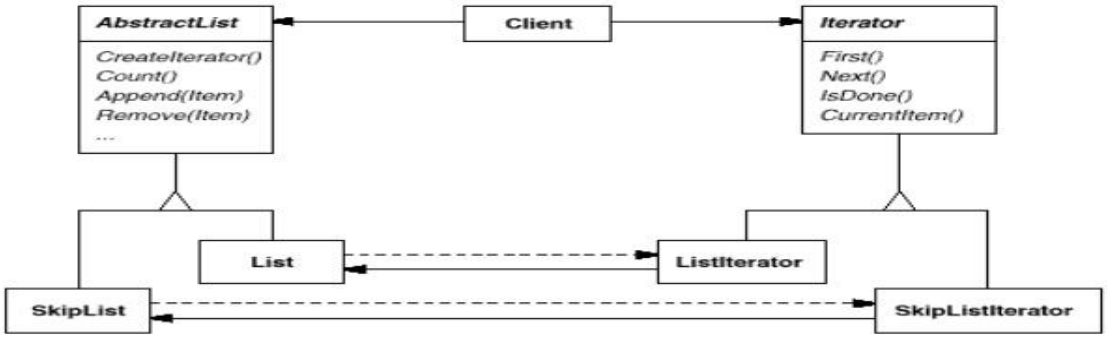
- Namena:
  - Sekvencijalno pristupanje elementima kolekcija bez uvida u unutrašnju reprezentaciju objekta
- Motivacija
  - Pristupanje elementima liste – obilazak
  - Klasa iterator definiše interfejs za pristupanje elementima liste
  - Podrška za više istovremenih obilazaka iste kolekcije
  - Jednoobrazan interfejs za obilaženje različitih kolekcija



Primer polimorfne iteracije



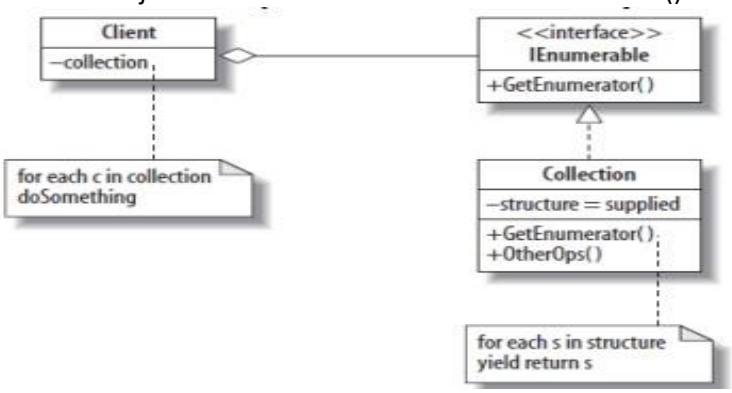
Struktura



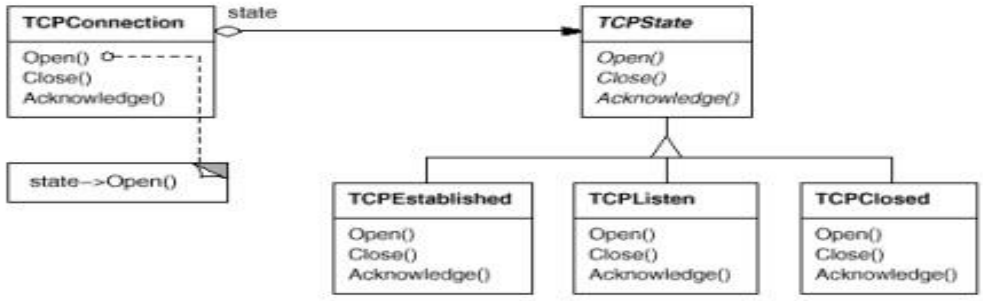
Učesnici
- Iterator - Definiše interfejs za pristup i obilaženje
- KonkretanIterator - Implementira interfejs Iterator
  - Prati tekuću poziciju u obilaženju kolekcije (Agregata)
- Agregat - Definiše interfejs za pravljenje objekta Iterator
- KonkretanAgregat - Implementira interfejs za pravljenje Iteratora i vraća primerak KonkretanIterator

Karakteristike
- Podržava varijacije za obilaženje kolekcija (npr. stabla)
- Iteratori pojednostavljuju interfejs Agregata
- Moguće je više istovremenih obilazaka (Iterator ima svoje stanje)

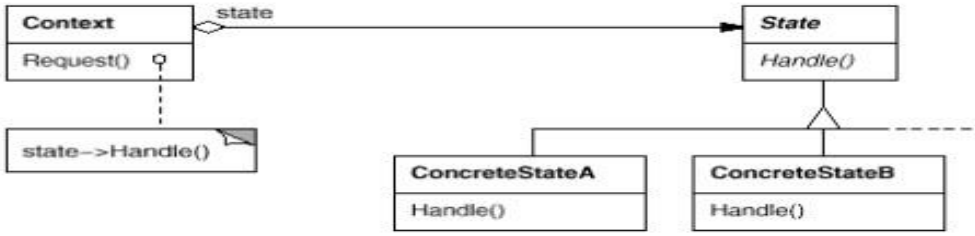
- U C#:
  - Interfejs IEnumerable sa metodom GetEnumerator()



42. Softverski obrazac –Stanje.
- Namena:
  - Omogućava da objekat promeni ponašanje kada mu se promeni unutrašnje stanje
- Motivacija:
  - Mrežna konekcija može biti u više stanja – uvodi se apstraktna klasa koja predstavlja stanje mrežne konekcije
  - TCPState deklariše zajednički Interfejs za sve klase stanja
  - Svaka klasa stanja implementira metode na poseban način
  - Kada se promeni stanje konekcije TCPConnection objekat menja objekat stanja



Struktura



Učesnici

• Kontekst - Definiše interfejs prema klijentima
- Održava primerak klase KonkretnoStanje (i delegira operacija u zavisnosti od stanja – može i sebe da prosledi u zahtevu ako stanje treba da mu pristupi)

• Stanje - Definiše interfejs za kapsuliranje ponašanja određenih stanja Konteksta
• KonkretnoStanje - Implementiraju ponašanje za određeno stanje Konteksta
• Odluku o prelasku u novo stanje može doneti Kontekst ili tekuće KonkretnoStanje

Karakteristike

• Lokalizacija ponašanje određeno stanjem u okviru jedne klase (izbegavanje glomaznih uslova)
• Eksplicitan prelazak iz stanja u stanje
• Tranziciju stanja može definisati Kontekst (ako su kriterijumi fiksni) ili potklasa klase Stanje
• Razmatranje načina pravljenja i uništavanja objekata Stanje
  • Po potrebi se prave i uništavaju kada nisu potrebni
  • Prave se unapred

Spajanje paterna

• Paterni se često koriste zajedno
• Kombinacija paterna podrazumeva istovremeno uključivanje u rešenje dva ili više paterna
• Postoje grupe paterna koje najčešće prate jedni druge
• Primer: Igrica – simulacija patkica

43. Refaktorisanje koda.

• Definicija: Refaktorisanje predstavlja izmenu interne strukture softvera u cilju lakšeg razumevanja i jednostavnije modifikacije, ali bez promene njegovog ponašanja
• Zadaci: Dodavanje funkcija i refaktorisanje
• Razumljiviji kod
• Lakše pronalaženje grešaka
• Brže pisanje koda
• 3R pravilo • Kada nešto radiš 1. put - RADI,
• kada radiš nešto slično 2. put – RAZMISLI,
• a kad radiš 3. put – REFAKTORIŠI

Proces refaktorisanja

- Zlatno pravilo: Ostavi kod u boljem stanju nego što si ga zatekao
  - Povećava efikasnost razvoja softvera
  - Mogu se razlikovati različiti režimi procesa refaktorisanja
  - Olakšano održavanje refaktorisanog softvera
  - Jedan od važnih preduslova - dobro testiranje
  - Razvoj vođen testovima (Test-Driven Development - TDD) se analizira u kontekstu refaktorisanja

Ciklusi razvoja vođenog testovima

- Dodavanje testa (test pada)
- Dodavanje funkcionalnosti (test prolazi – ne razmišlja se o strukturi koda)
- Refaktorisanje (sređivanje koda)

Koncept dva šešira

Refaktorisanje
- Male promene uz očuvanje ponašanja
- Ostaju zeleni testovi
- Preporučuje se konstantno testiranje refaktorisanog koda
- Cilj: poboljšanje dizajna
- Razumljiviji kod

Dodavanje funkcija
- Uzrokuje dodavanje novih testova
- Preporučuju se male izmene koje bi brzo dovele do zelenih testova
- Dominantno na početku razvoja

44. Tipovi refaktorisanja koda

Refaktorisanje tipa - „Čišćenje smeća“

- Čišćenje koda od očiglednog smeća
- Neki od razloga za postojanje takvog koda:
- Stalno dodavanje funkcionalnosti
- Neiskusan programer
- „loš dan“ za programiranje
- „Možda bi mogao nešto kasnije uraditi“
- Čišćenjem koda omogućuje brže i jednostavnije dodavanje novih funkcionalnosti
- Naziva se i pravilo izviđača – uvek ostaviti u boljem stanju od zatečenog

Refaktorisanje sa razumevanjem

- Šta radi neki blok koda?
- Promena imena klasama , metodama ili promenljivama.
- Restruktuiranje koda radi lakšeg razumevanja.
- Izdvajanje blokova u posebne metode.
- Dobro je konsultovati se sa nekim drugim (ako može sa autorom)
- Razumevanje – pojašnjavanje – poboljšanje.

Tok akcija prilikom refaktorisanja

- Dele ga oba tipa refaktorisanja: čišćenje smeća i razumevanje koda
- Uočavaju problem radeći nešto drugo

Tekuća verzija koda (ranije napisana) može biti nepotpuna i trebalo bi da se popravi

Dobro je odmah rešiti problem u kodu ako je:

- jednostavno odraditi ili
- lakše dodati novu funkcionalnost

Važno je refaktorisati kod da se dobije stabilna baza u kojoj svi testovi prolaze (zeleni su). Može se koristiti sistem upravljanja verzijama.

Refaktorisati problematičan kod Ako refaktorisanje traje duže od predviđenog, skloniti izmene i vratiti ih kasnije

Ako refaktorisanje deluje previše zahtevno da se odmah radi, raditi ga nakon dodavanja funkcionalnosti.

Nije dovoljno samo dodati novu funkcionalnost, već i ostaviti čist kod (ili bar čistiji od zatečenog)

Pripremno refaktorisanje

- Kada se krene sa dodavanjem nove funkcionalnosti i utvrdi da struktura koda nije adekvatna
- Tada se vrši refaktorizacija
- Nakon izmene strukture olakšano je dodavanje funkcionalnosti
- Isplati se ako je ukupna promena koda nego samo dodavanje funcionalnosti
- Analogija sa farbanjem sa i bez pripreme materijala

Pripremno refaktorisanje se može uraditi kada se dodaje nova funkcionalnost

Planirano refaktorisanje

- Planira se u okviru posebnih „user story“-ja
- Koristi se za popravljanje većih celina u „problematičnom“ kodu
- Ponekad se unapred definiše način refaktorisanje (izmena interfejsa i sl.)
- Pokazuju da nije dovoljno vremena posvećeno ranijem razvoju

Dugotrajno refaktorisanje

- Kada je neophodno izmeniti relacije i takve izmene zahtevaju više meseci
- Restrukturiranje koje zahteva izmene veće od jednog razvojnog ciklusa
    - npr. izmena modela podataka,
    - mehanizma komunikacije između komponenti i sl.
- Usaglašavanje ciljeva tima
- Može se iskoristiti tehnika apstraktnih grana – apstraktni sloj podržava i tekuću i izmenjenu verziju koda

Problemi sa refaktoriranjem

- Objasniti nadređenom
- Menjati model baze podataka
- Menjati interfejse – prvo dobro osmisliti pa tek onda objaviti
- Ako je kod jako „zbrkan“ bolje je pisati ga ispočetka
- U slučaju kratkih rokova za isporuku – izostaviti refaktorisanje
- Refaktorisanje ↔ performanse

Različiti pristupi u refaktorisanju

1. raspodela vremena između komponente – svakoj komponenti se ograničavaju resursi
2. pristup stalne optimizacije koda - 90% koda ne treba optimizovati (retko se poziva), a samo mali deo koda troši mnogo vremena
3. optimizacija profajliranjem programa

• Potrebno je balansirati refaktorisanje koda sa proširenjem funkcionalnosti
• Voditi računa o ekonomskoj opravdanosti refaktorisanja

Osnovni koraci

1. Počistiti nered - sve u kodu što nema dodeljenu vrednost
   • Formatiraj kod • Izbrisati komentare • Izbrisati mrtav kod • Izbrisati nepotreban kod
2. Eliminisati nepotrebnu složenost
   • Loša imena • Duge metode • Duboke uslovne petlje • Magične brojeve • Neodgovarajuće opsege promenljivih • Nerazumljive blokove